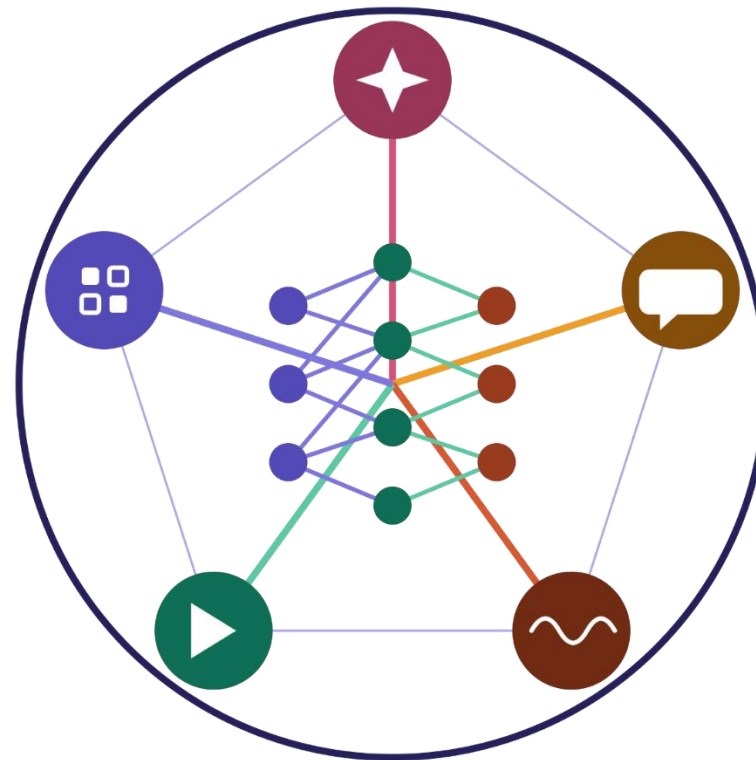


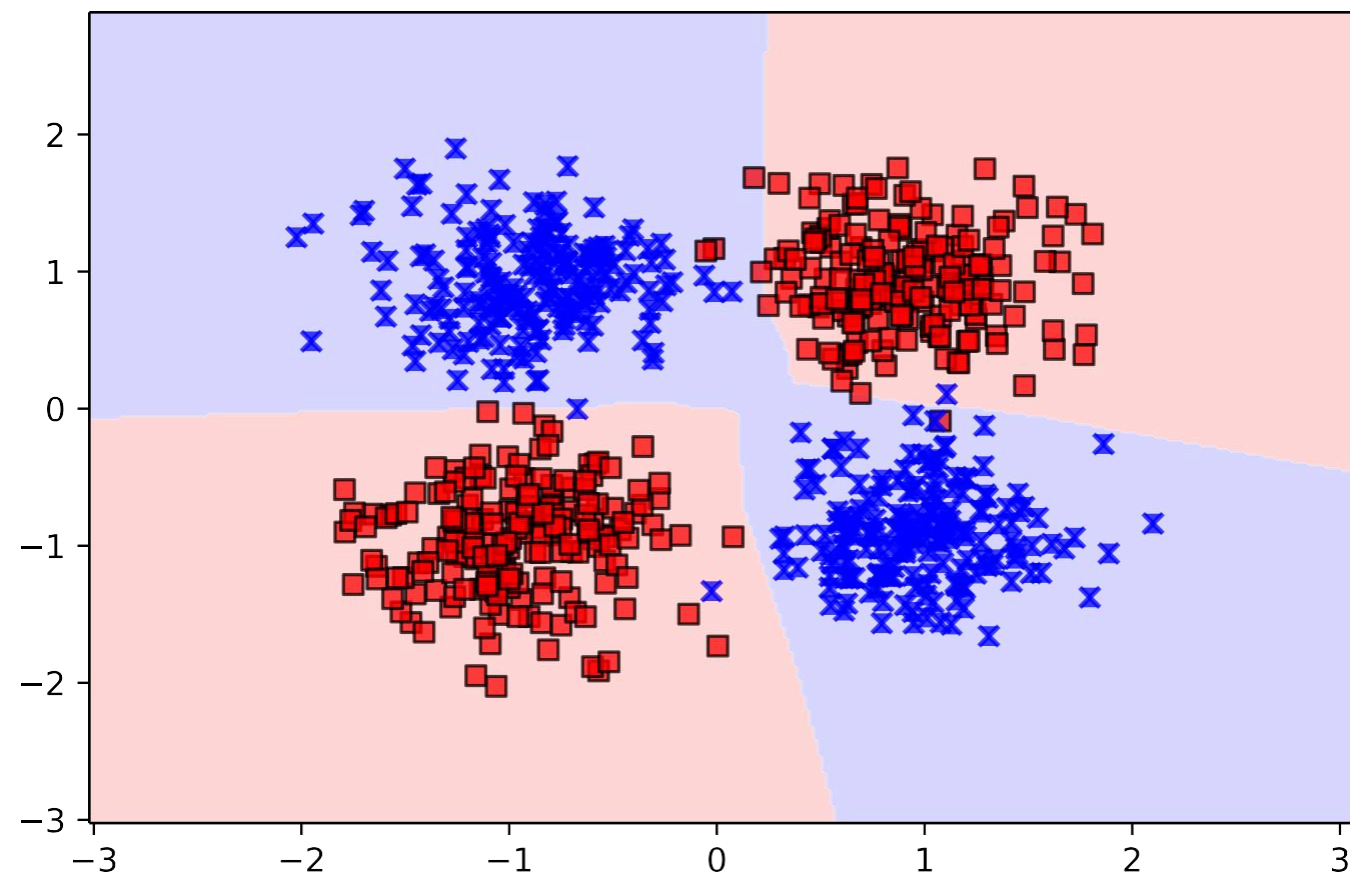


Lecture 07

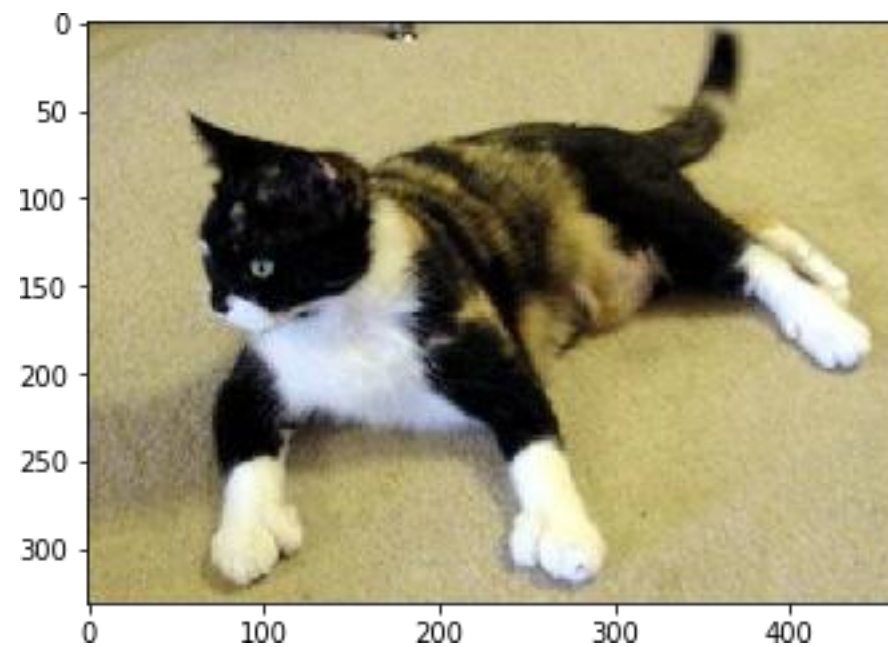
Multilayer Perceptrons

Today

We will finally be able to solve the XOR problem ...



... and talk about cats!



Lecture Overview

1. Multilayer Perceptron Architecture
2. Nonlinear Activation Functions
3. Multilayer Perceptron Code Examples
4. Overfitting and Underfitting
5. Cats & Dogs and Custom Data Loaders

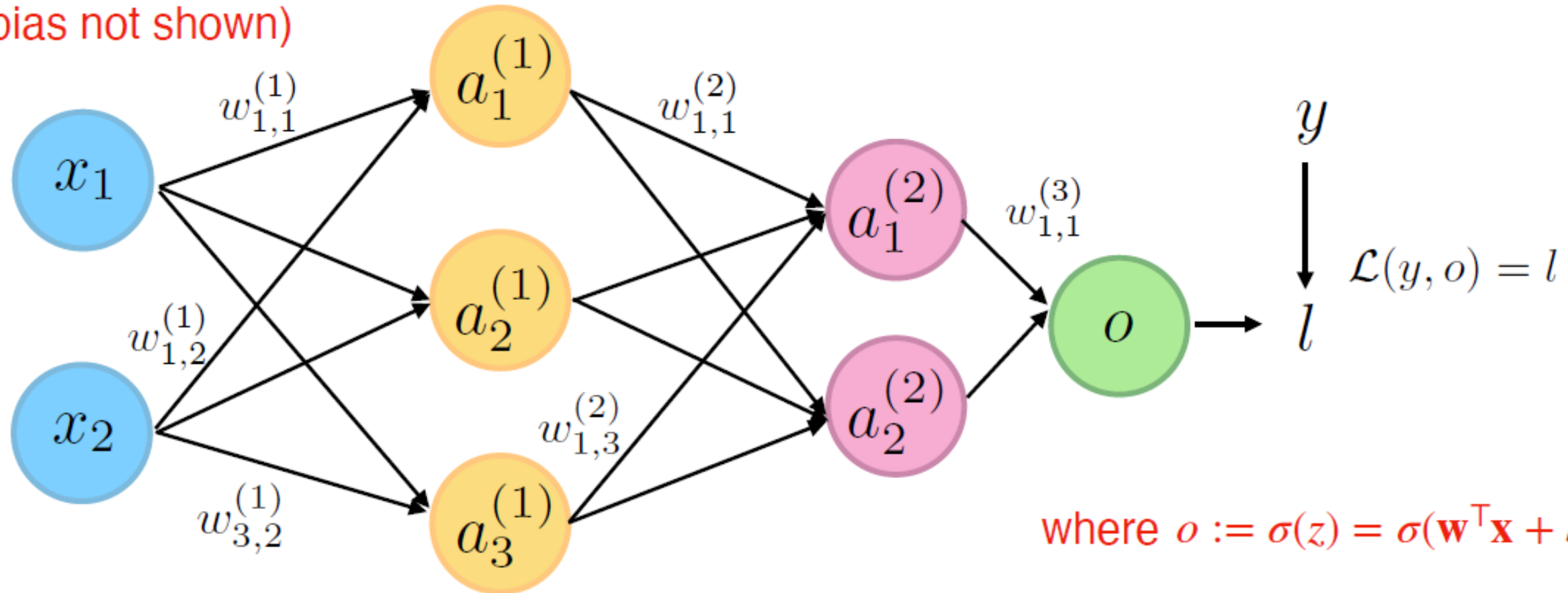
Fully-connected feedforward neural networks with one or more hidden layers

- 1. Multilayer Perceptron Architecture**
2. Nonlinear Activation Functions
3. Multilayer Perceptron Code Examples
4. Overfitting and Underfitting
5. Cats & Dogs and Custom Data Loaders

Computation Graph with Multiple Fully-Connected Layers = Multilayer Perceptron

Nothing new, really

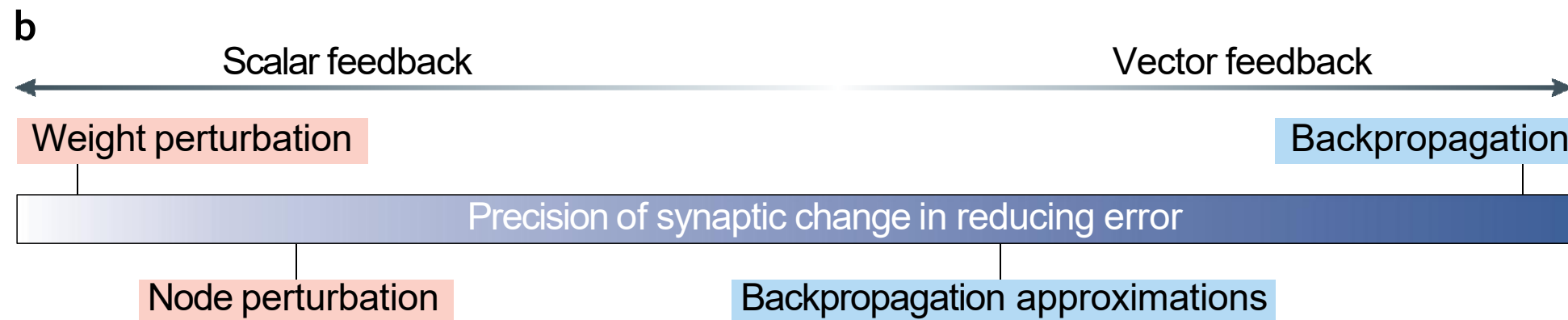
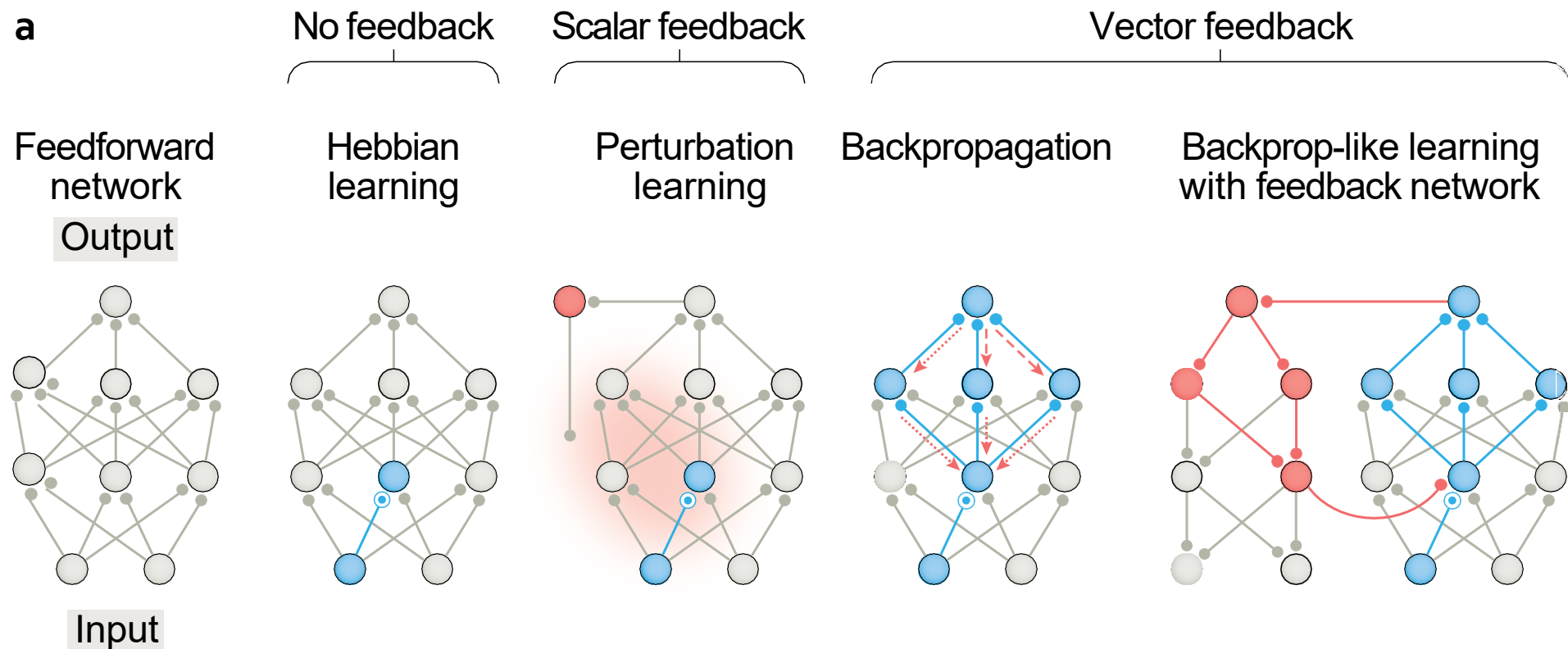
(bias not shown)



where $o := \sigma(z) = \sigma(\mathbf{w}^T \mathbf{x} + b)$

(Assume network for binary classification)

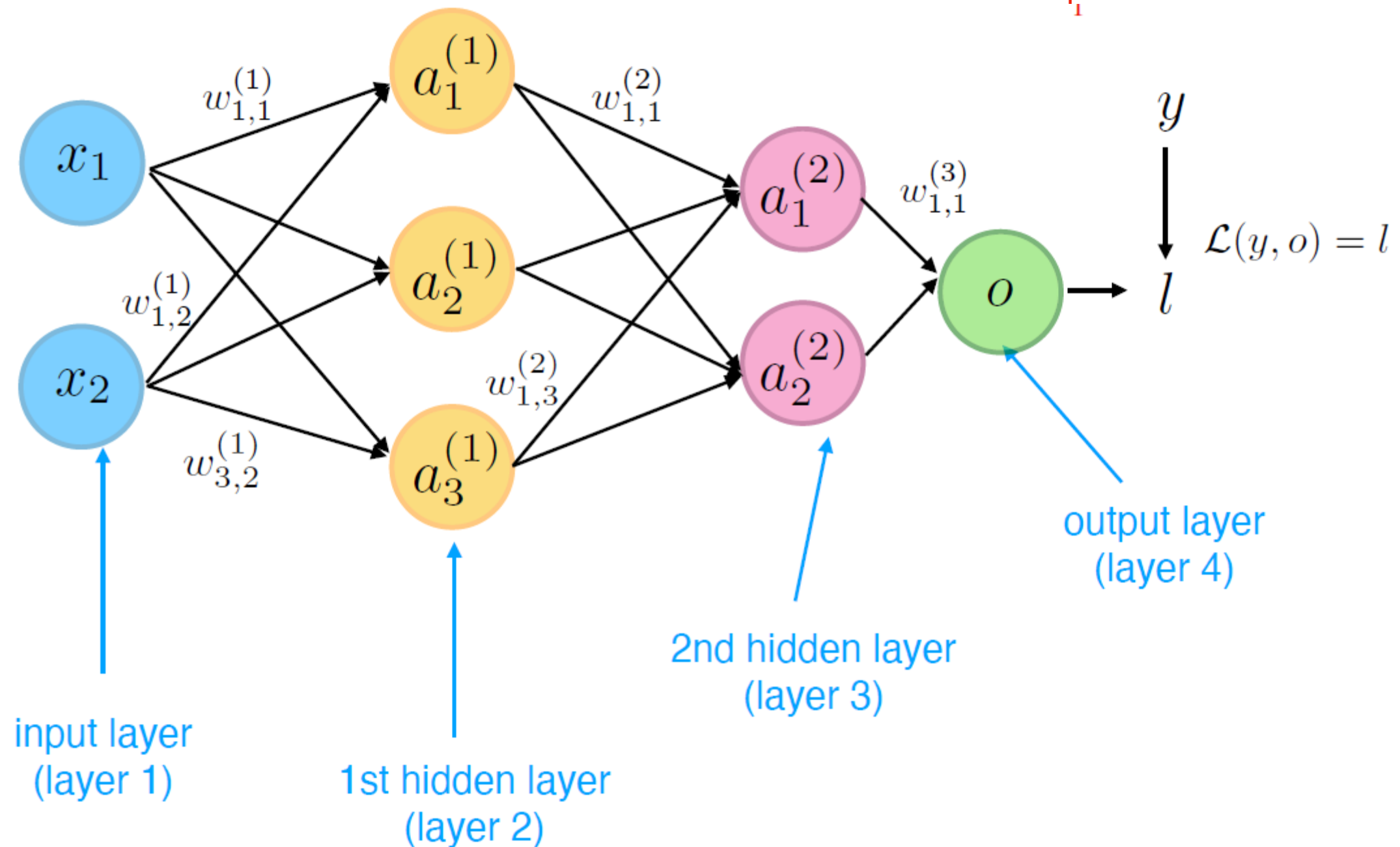
$$\begin{aligned} \frac{\partial l}{\partial w_{1,1}^{(1)}} &= \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}} \\ &\quad + \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}} \end{aligned}$$



Lillicrap, T. P., Santoro, A., Marris, L., & Akerman, C. (n.d.). Backpropagation and the brain. *Nature Reviews Neuroscience*, 1–12. <https://doi.org/10.1038/s41583-020-0277-3>

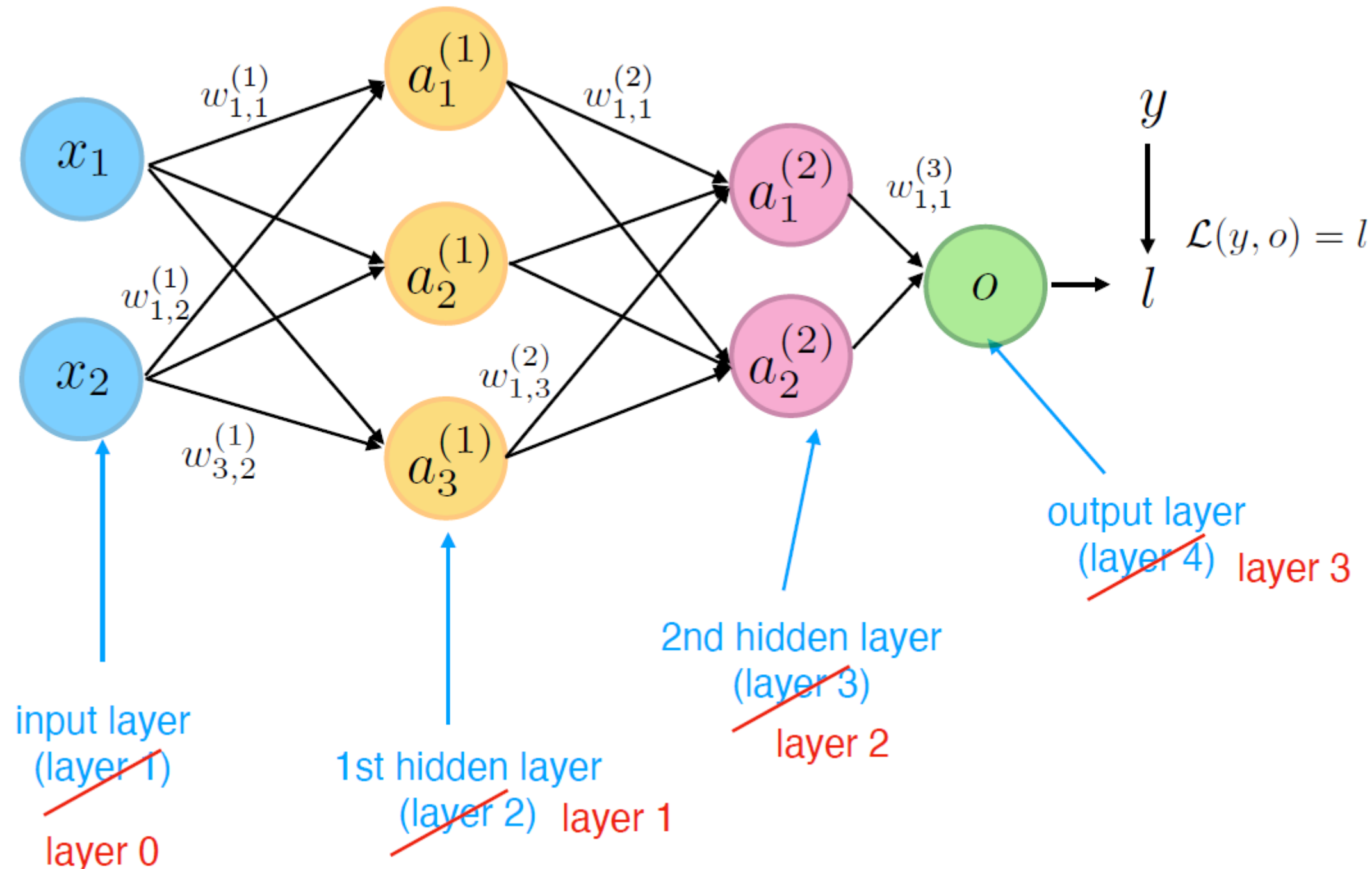
Computation Graph with Multiple Fully-Connected Layers = Multilayer Perceptron

here, we could also write O
as $a_1^{(3)}$

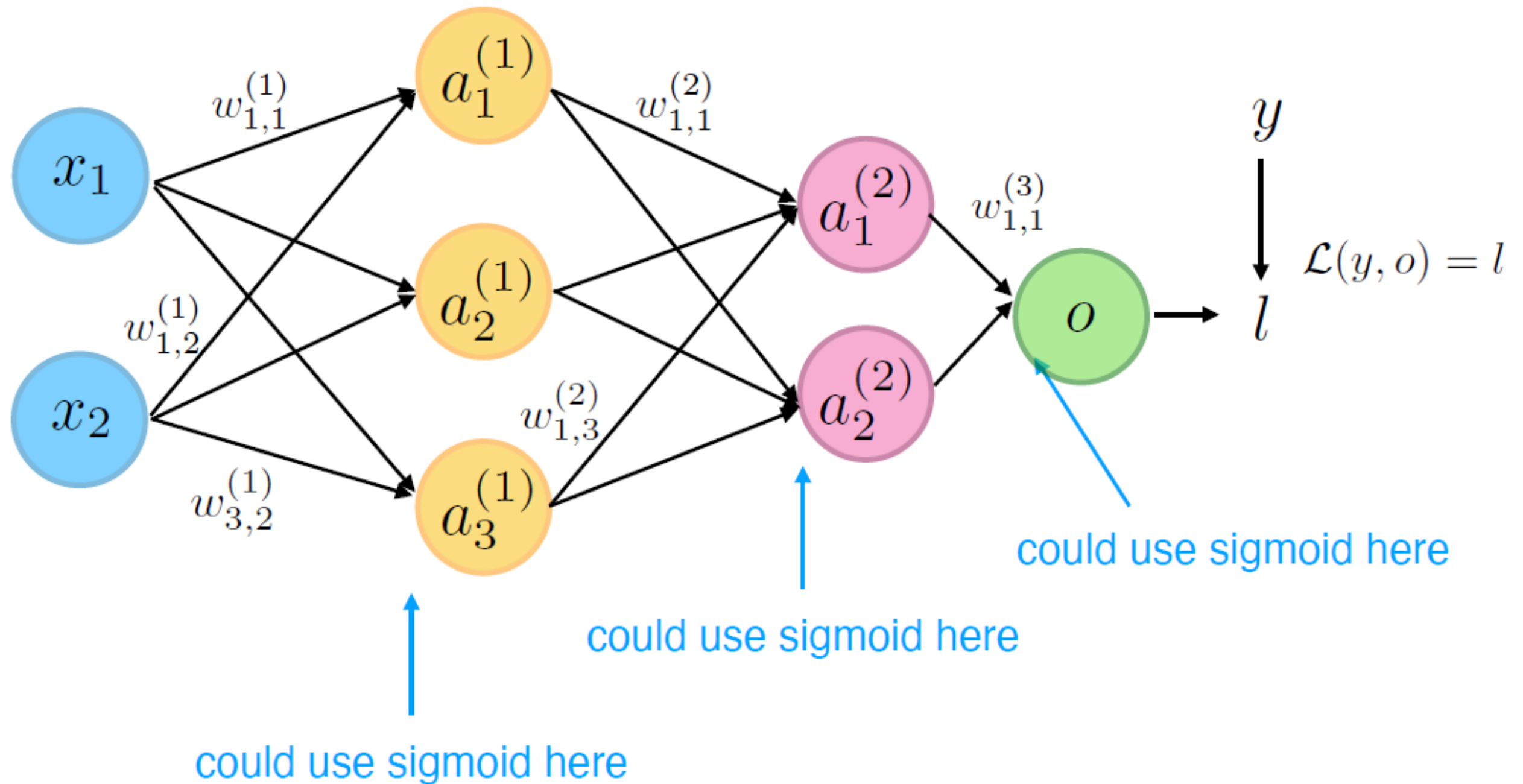


Computation Graph with Multiple Fully-Connected Layers = Multilayer Perceptron

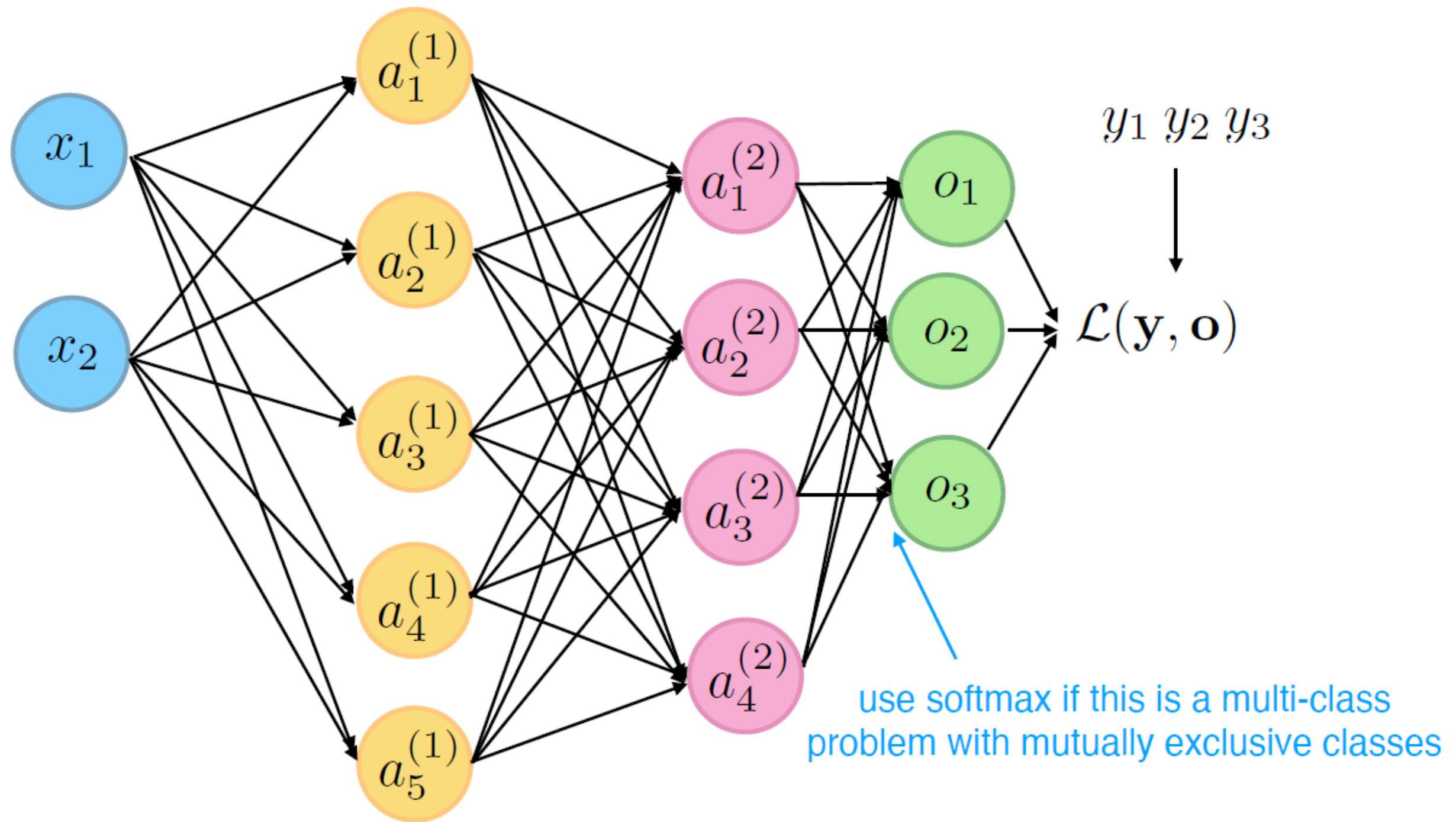
A more common counting/naming scheme, because then a perceptron/Adaline/
logistic regression model can be called a "1-layer neural network"



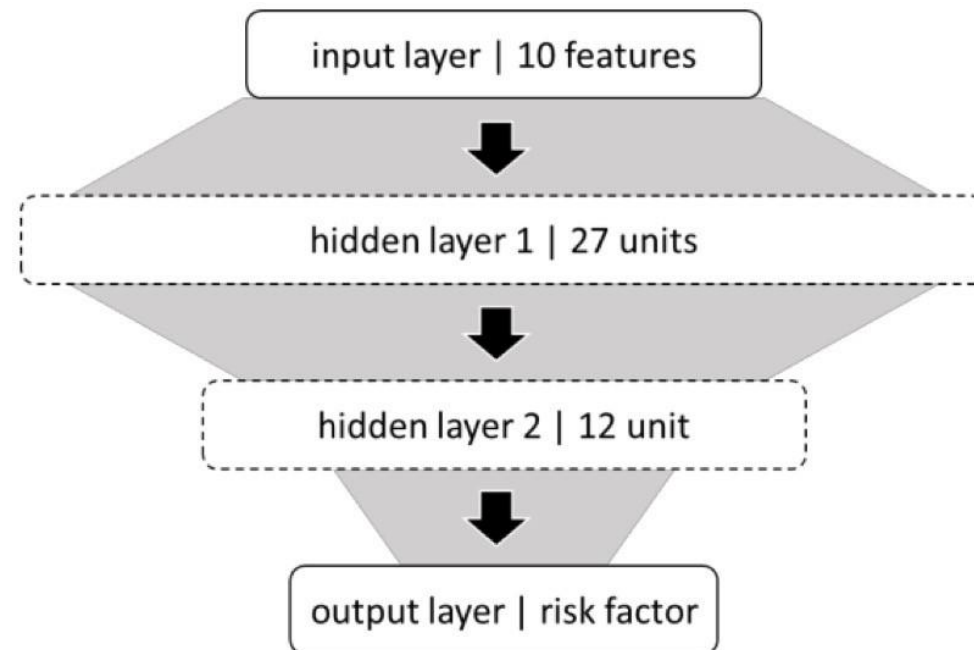
Computation Graph with Multiple Fully-Connected Layers = Multilayer Perceptron



Computation Graph with Multiple Fully-Connected Layers = Multilayer Perceptron



Is It Deep Learning?



Supplementary Figure 2. Illustration of the feed forward neural network architecture of the proposed deep survival model.

Note That the Loss is Not Convex Anymore

- Linear regression, Adaline, Logistic Regression, and Softmax Regression have convex loss functions
- This is not the case anymore; in practice, we usually end up at different local minima if we repeat the training (e.g., by changing the random seed for weight initialization or shuffling the dataset while leaving all settings the same)
- In practice though, we WANT to explore different starting weights, however, because some lead to better solutions than others

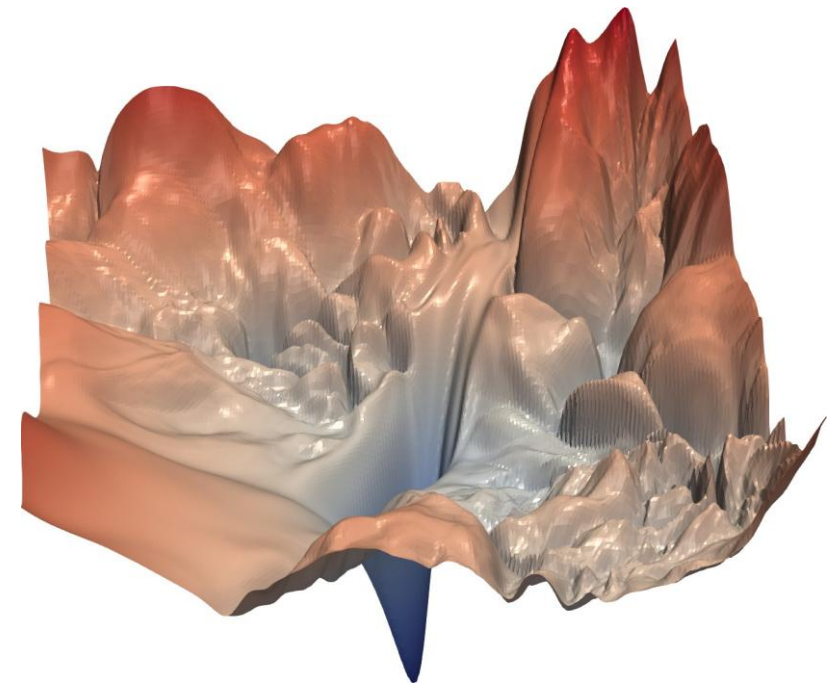


Image Source: Li, H., Xu, Z., Taylor, G., Studer, C. and Goldstein, T., 2018. Visualizing the loss landscape of neural nets. In *Advances in Neural Information Processing Systems* (pp. 6391-6401).

About Softmax & Sigmoid in the Output Layer and Issues with MSE

- Sigmoid activation + MSE has the problem of very flat gradients when the output is very wrong i.e., 10^{-5} probability and class label 1

$$\frac{\partial \mathcal{L}}{\partial w_j} = -\frac{2}{n}(\mathbf{y} - \mathbf{a}) \odot \sigma(\mathbf{z}) \odot (1 - \sigma(\mathbf{z})) \mathbf{x}_j^\top \quad (\text{derivative for sigmoid + MSE neuron})$$

- Softmax (forces network to learn probability distribution over labels) in output layer is better than sigmoid because of the mutually exclusive labels as discussed in the Softmax lecture; hence, in output layer, softmax is usually better than sigmoid

What happens if we initialize the multilayer perceptron to all-zero weights?

Complex, nonlinear decision boundaries with hidden layer and nonlinear activations

1. Multilayer Perceptron Architecture

2. Nonlinear Activation Functions

3. Multilayer Perceptron Code Examples

4. Overfitting and Underfitting

5. Cats & Dogs and Custom Data Loaders

PyTorch Recap

```
import torch.nn.functional as F
```

```
class MultilayerPerceptron(torch.nn.Module):
```

```
    def __init__(self, num_features, num_classes):
        super(MultilayerPerceptron, self).__init__()

        ### 1st hidden layer
        self.linear_1 = torch.nn.Linear(num_features,
                                         num_hidden_1)

        ### 2nd hidden layer
        self.linear_2 = torch.nn.Linear(num_hidden_1,
                                         num_hidden_2)

        ### Output layer
        self.linear_out = torch.nn.Linear(num_hidden_2,
                                           num_classes)

    def forward(self, x):
        out = self.linear_1(x)
        out = F.relu(out)
        out = self.linear_2(out)
        out = F.relu(out)
        logits = self.linear_out(out)
        probas = F.log_softmax(logits, dim=1)
        return logits, probas
```

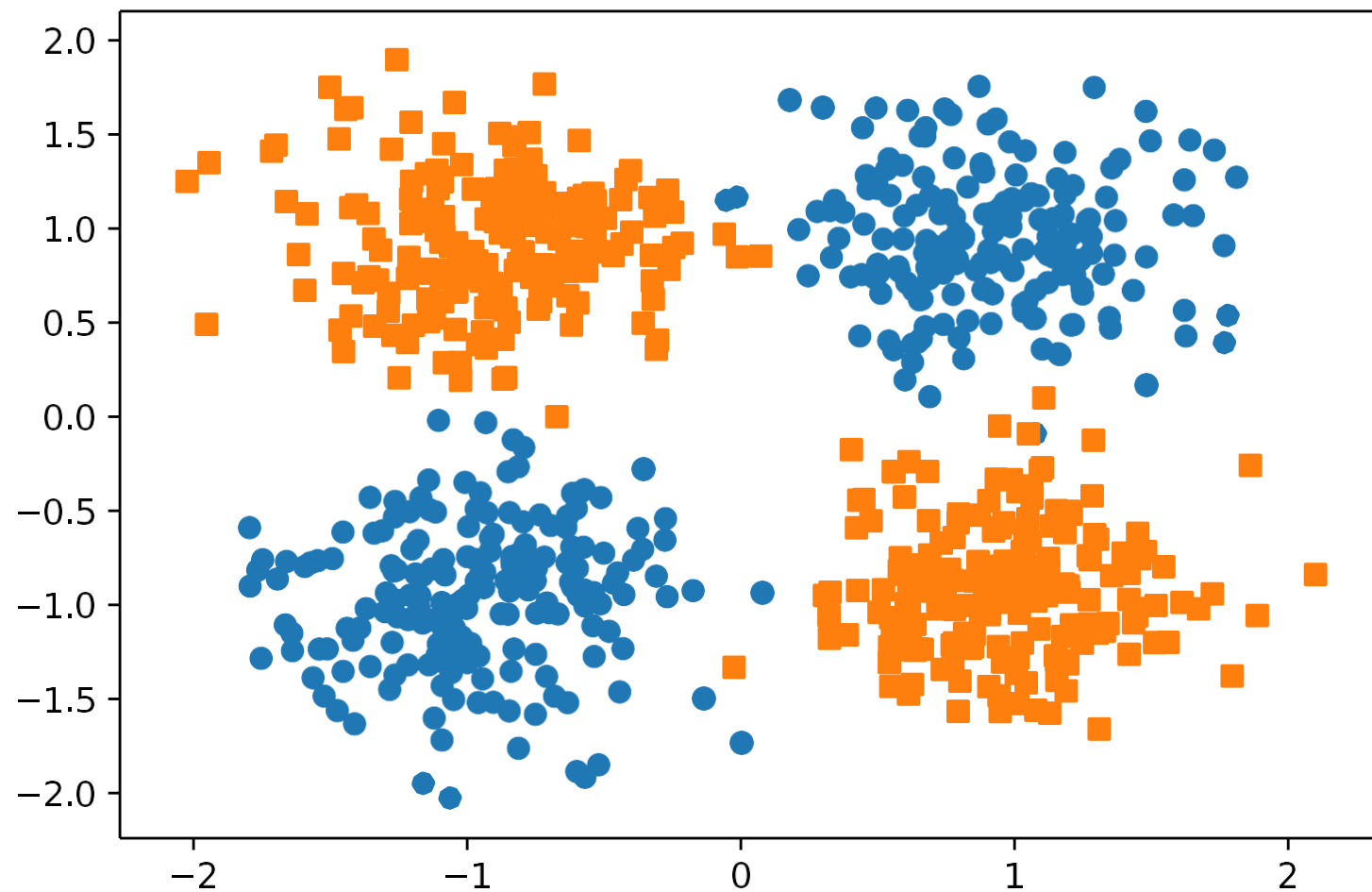
```
class MultilayerPerceptron(torch.nn.Module):
```

```
    def __init__(self, num_features, num_classes):
        super(MultilayerPerceptron, self).__init__()

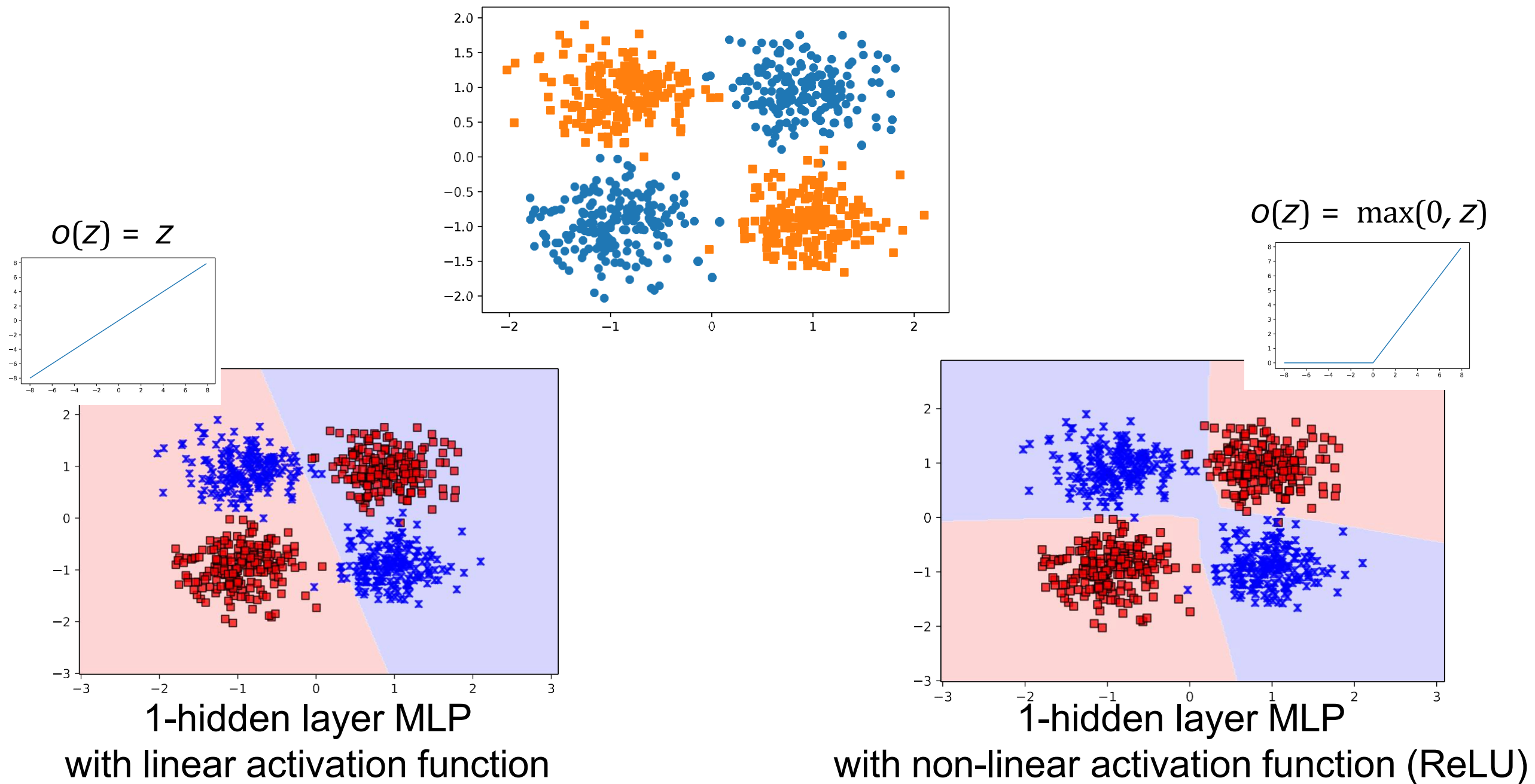
        self.my_network = torch.nn.Sequential(
            torch.nn.Linear(num_features, num_hidden_1),
            torch.nn.ReLU(),
            torch.nn.Linear(num_hidden_1, num_hidden_2),
            torch.nn.ReLU(),
            torch.nn.Linear(num_hidden_2, num_classes)
        )

    def forward(self, x):
        logits = self.my_network(x)
        probas = F.softmax(logits, dim=1)
        return logits, probas
```

Solving the XOR Problem with Non-Linear Activations



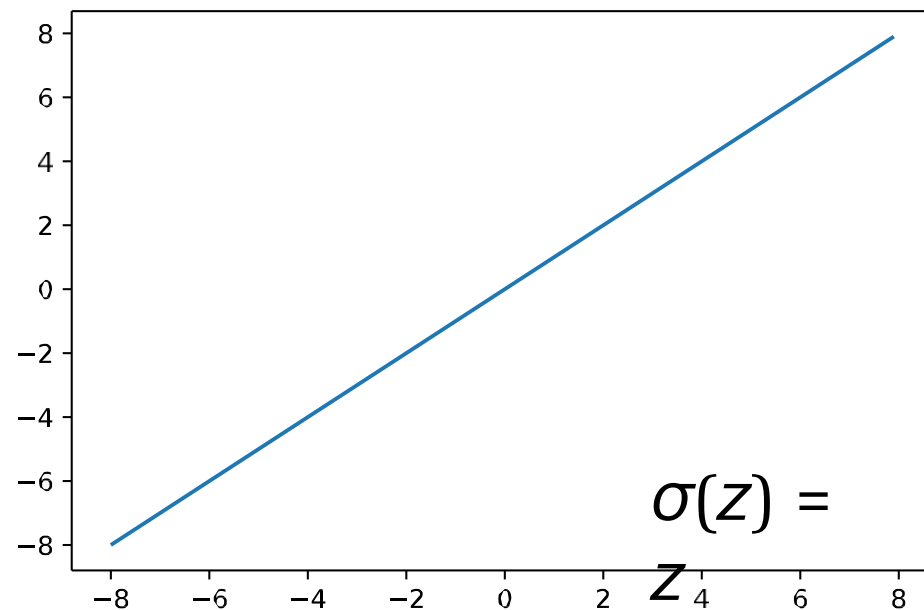
Solving the XOR Problem with Non-Linear Activations



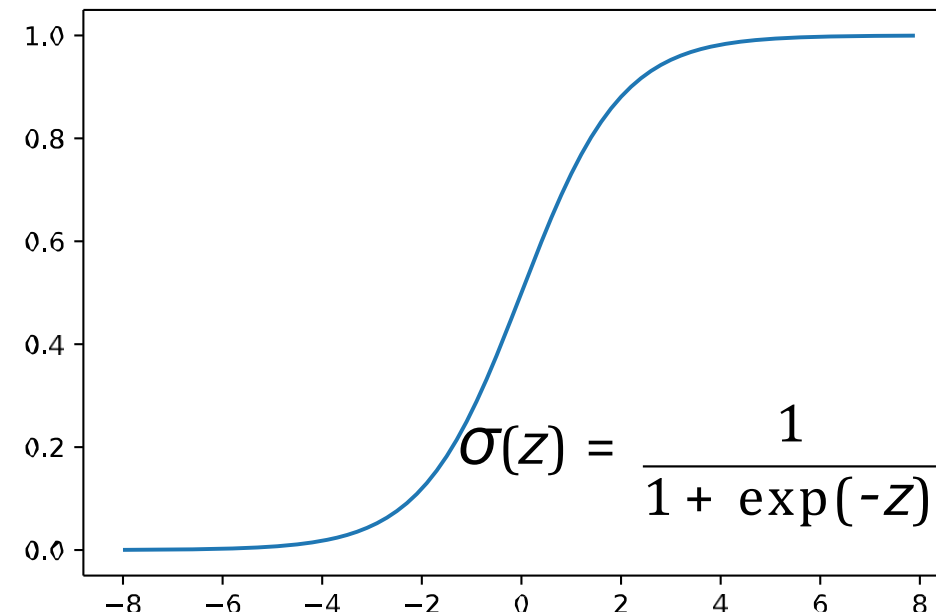
<https://github.com/rasbt/stat453-deep-learning-ss21/blob/master/L09/code/xor-problem.ipynb>

A Selection of Common Activation Functions (1)

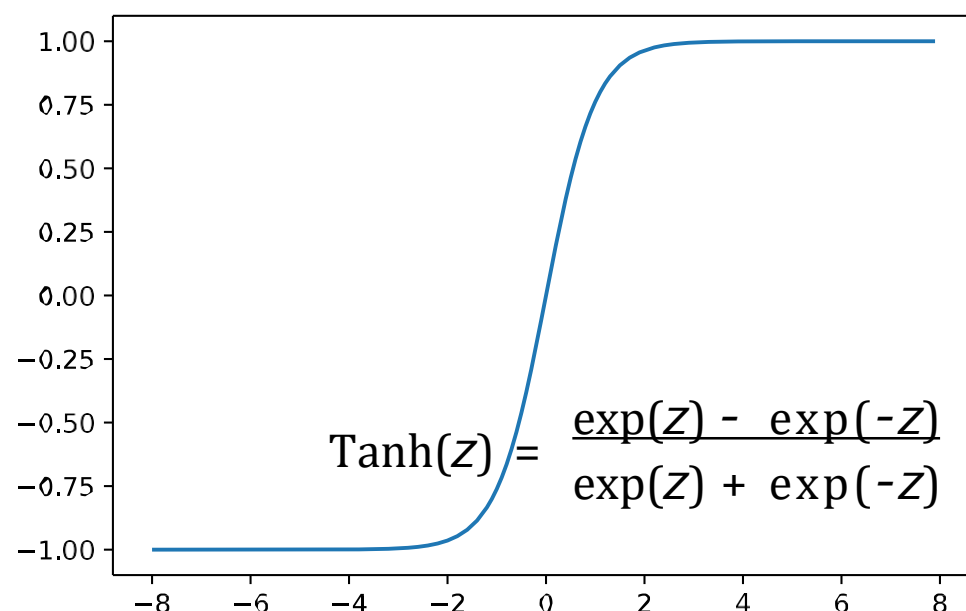
Identity



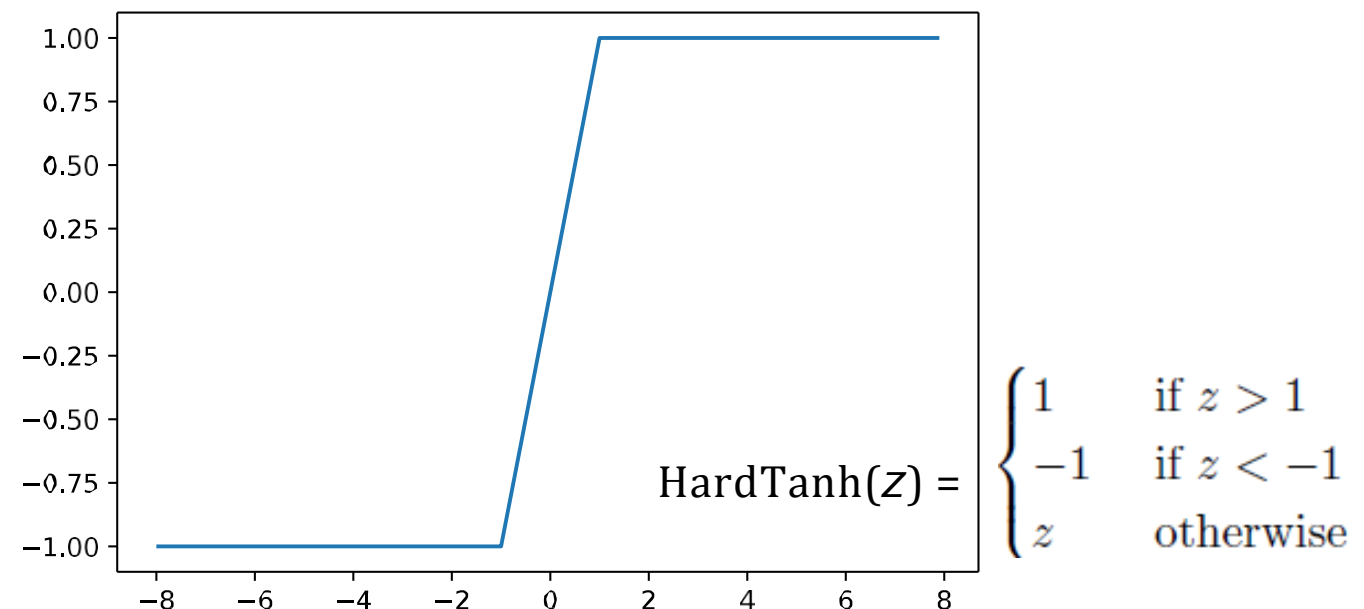
(Logistic) Sigmoid



Tanh ("tanH")

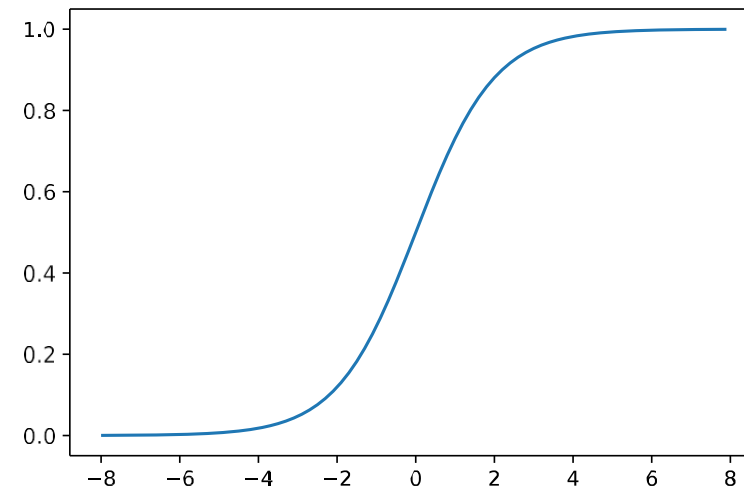


Hard Tanh



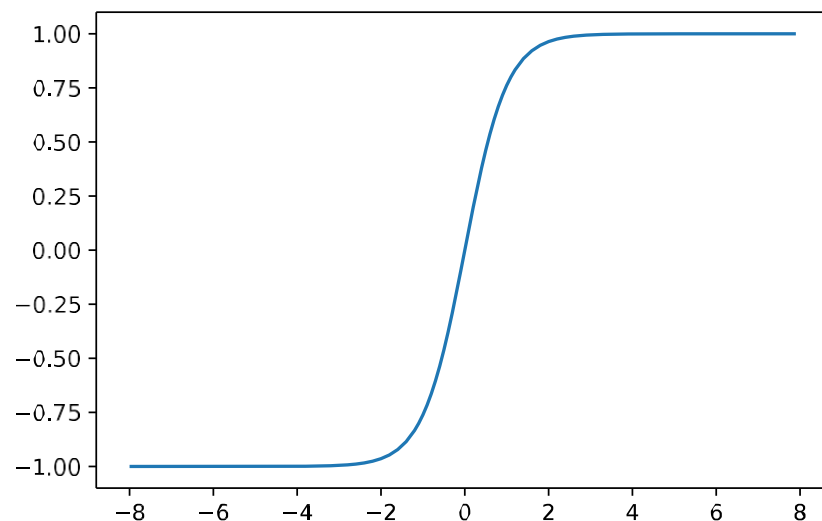
A Selection of Common Activation Functions (1)

(Logistic) Sigmoid



- Advantages of Tanh
- Mean centering
- Positive and negative values
- Larger gradients

Tanh ("tanH")



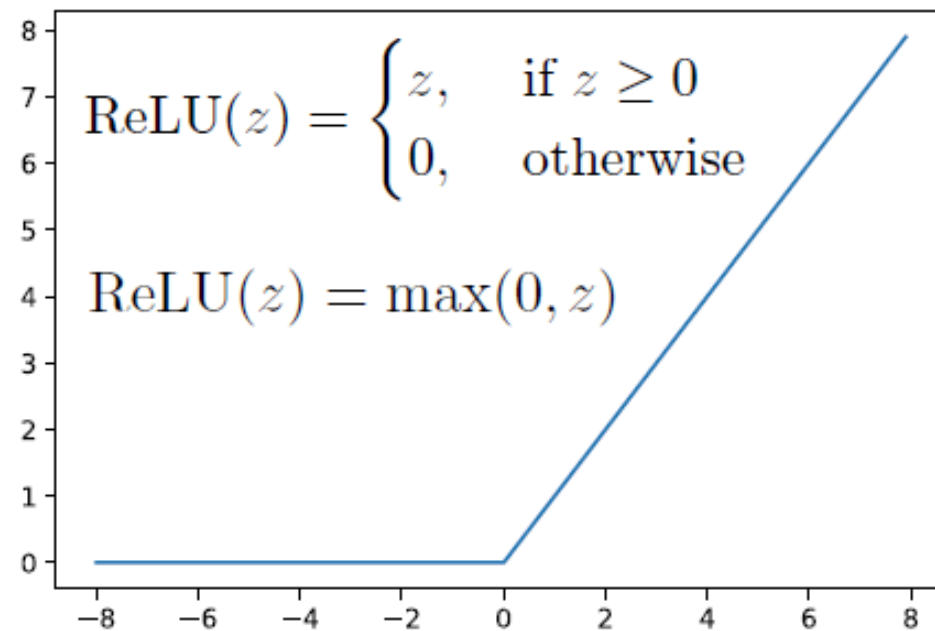
Additional tip: Also good to normalize inputs to mean zero and use random weight initialization with avg. weight centered at zero

Also simple derivative:

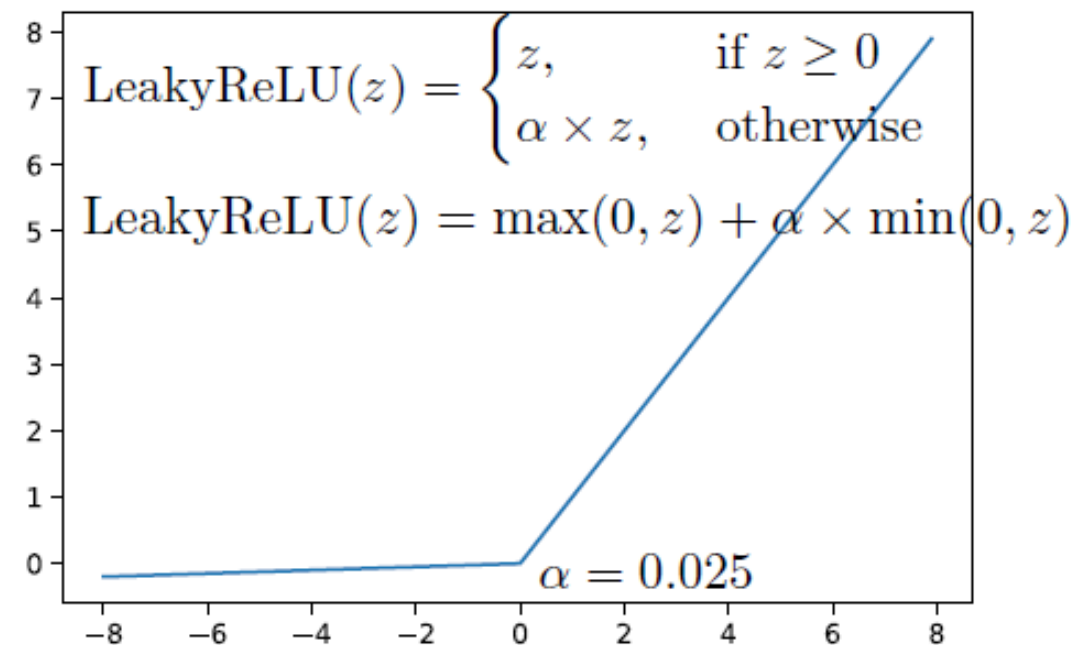
$$\frac{d}{dz} \text{Tanh}(z) = 1 - \text{Tanh}(z)^2$$

A Selection of Common Activation Functions (2)

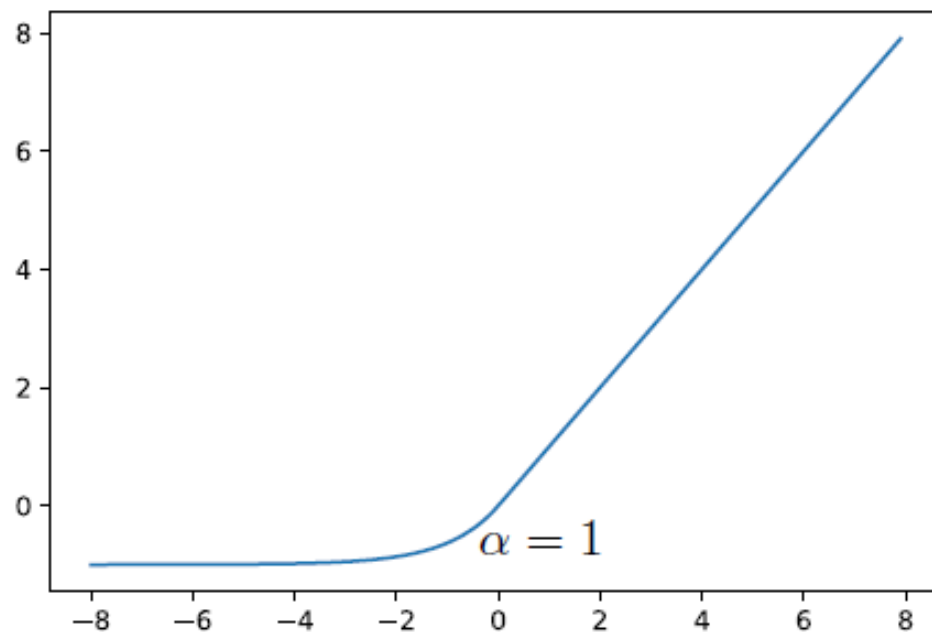
ReLU (Rectified Linear Unit)



Leaky ReLU



ELU (Exponential Linear Unit)



PReLU (Parameterized Rectified Linear Unit)

here, alpha is a trainable parameter

$$\text{PReLU}(z) = \begin{cases} z, & \text{if } z \geq 0 \\ \alpha z, & \text{otherwise} \end{cases}$$

$$\text{PReLU}(z) = \max(0, z) + \alpha \times \min(0, z)$$

Smooth Adversarial Training

Cihang Xie, Mingxing Tan, Boqing Gong, Alan Yuille, Quoc V. Le

<https://arxiv.org/abs/2006.14536>

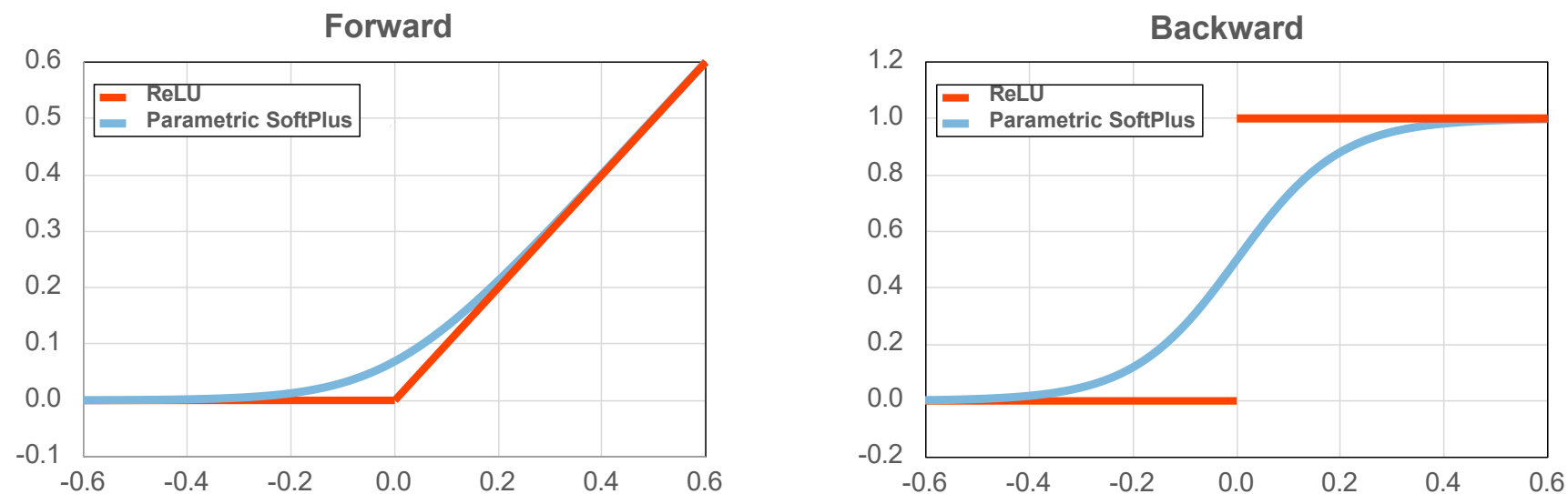


Figure 1: Left Panel: ReLU and Parametric SoftPlus. Right Panel: the first derivatives for ReLU and Parametric SoftPlus. Compared to ReLU, Parametric Softplus is smooth with continuous derivatives.

It is commonly believed that networks cannot be both accurate and robust, that gaining robustness means losing accuracy. [...] Our key observation is that the widely-used ReLU activation function significantly weakens adversarial training due to its non-smooth nature. Hence we propose *smooth adversarial training (SAT)*, in which we replace ReLU with its smooth approximations to strengthen adversarial training.

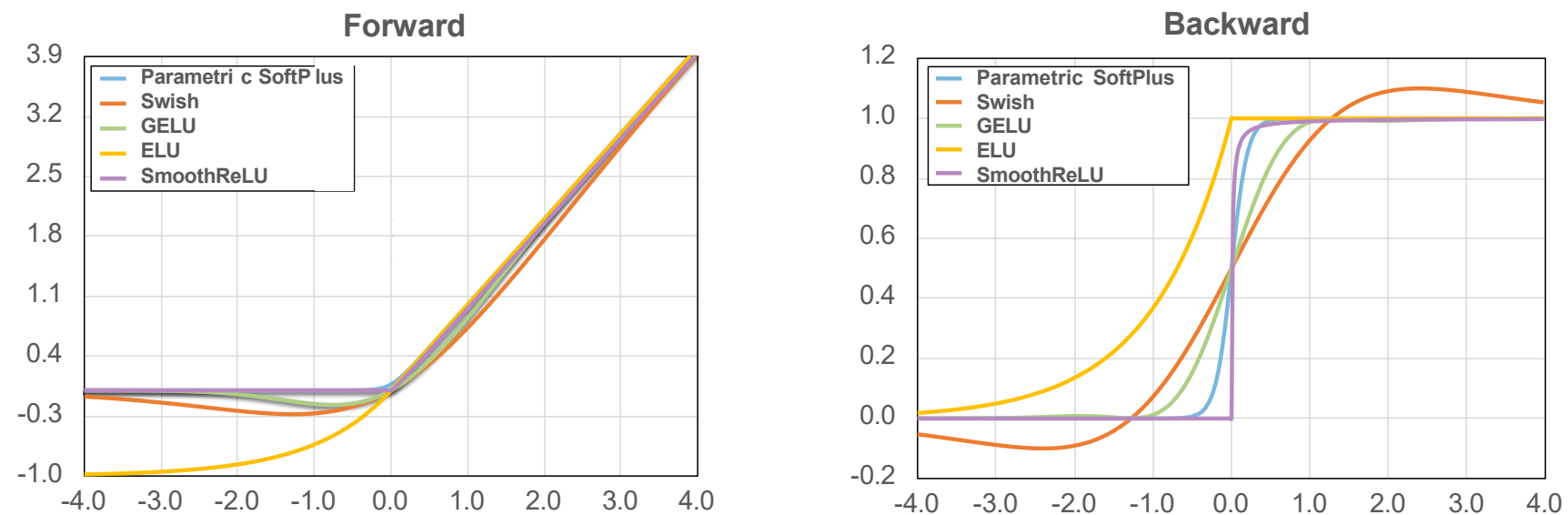


Figure 2: Visualizations of smooth activation functions and their derivatives.

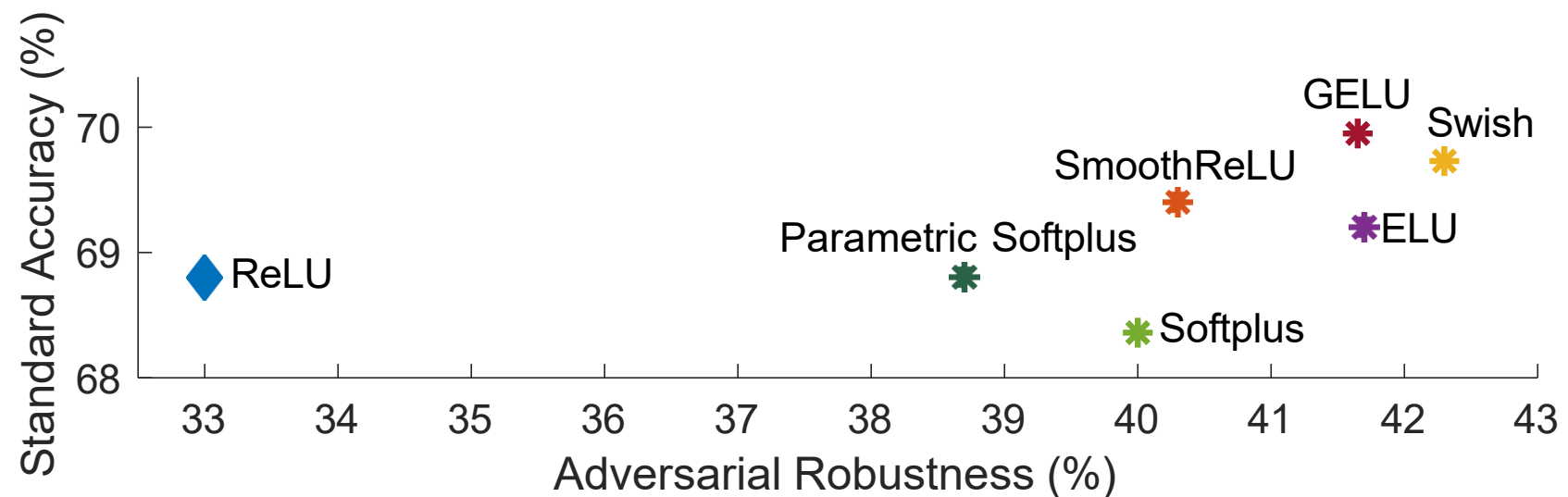


Figure 3: Smooth activation functions improve adversarial training. Compared to ReLU, all smooth activation functions significantly boost robustness, while keeping accuracy almost the same.

<https://arxiv.org/abs/2006.14536>

Dance Moves of Deep Learning Activation Functions

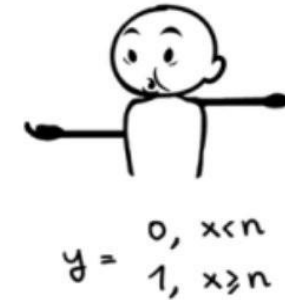
Sigmoid



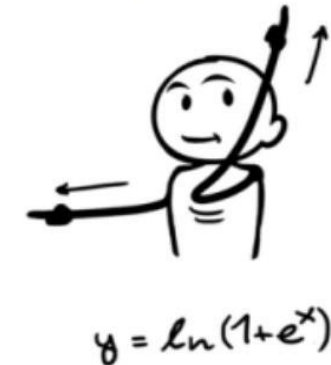
Tanh



Step Function



Softplus



source: sefiks

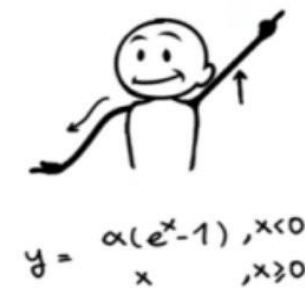
ReLU



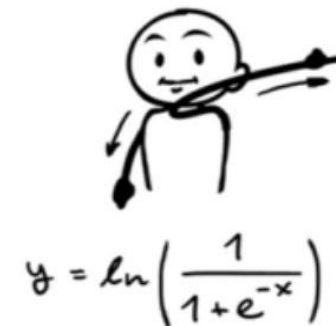
Softsign



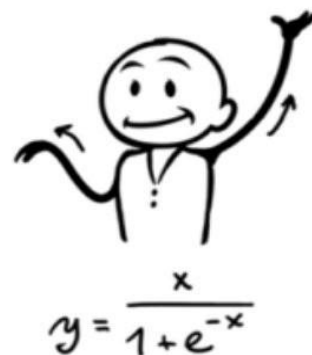
ELU



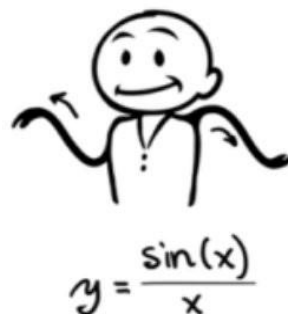
Log of Sigmoid



Swish



Sinc



Leaky ReLU



Mish



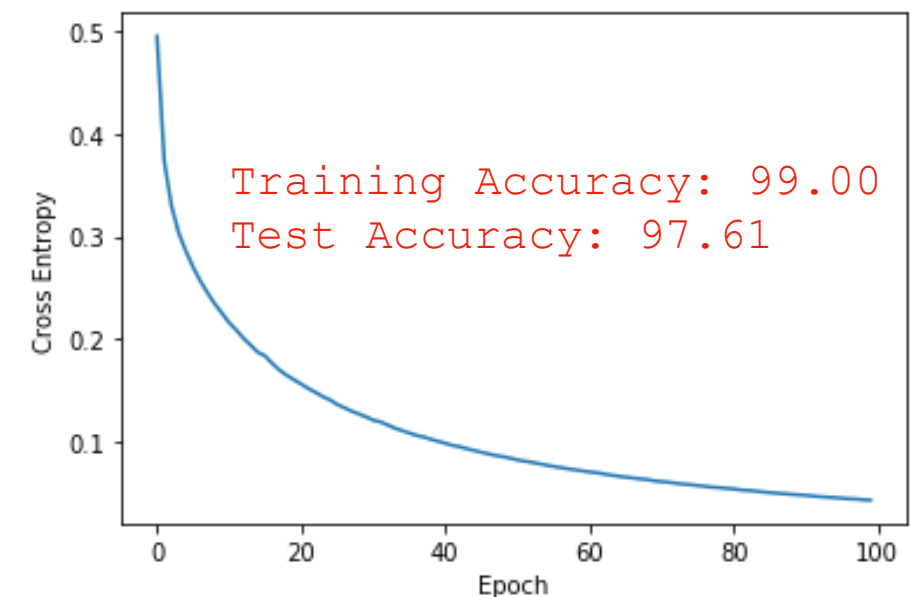
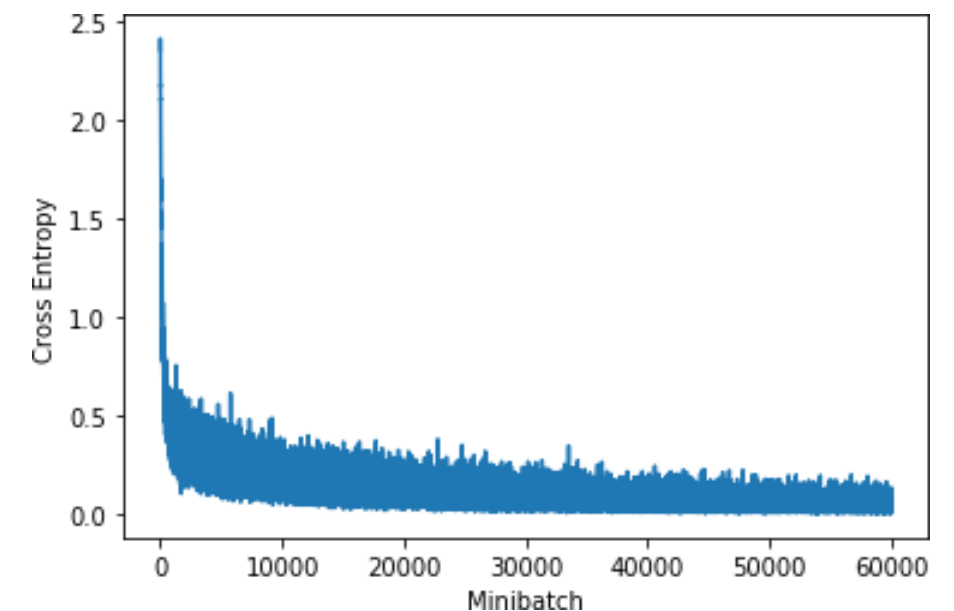
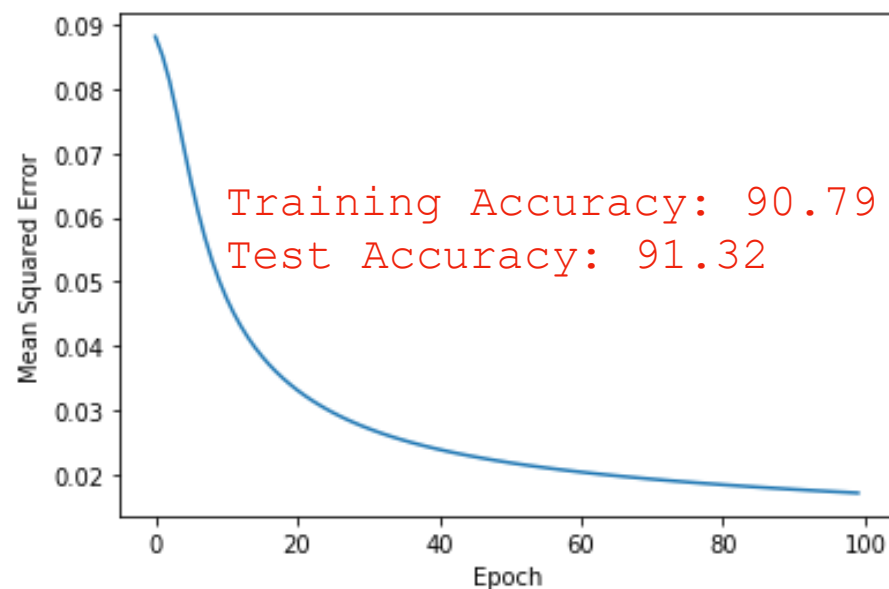
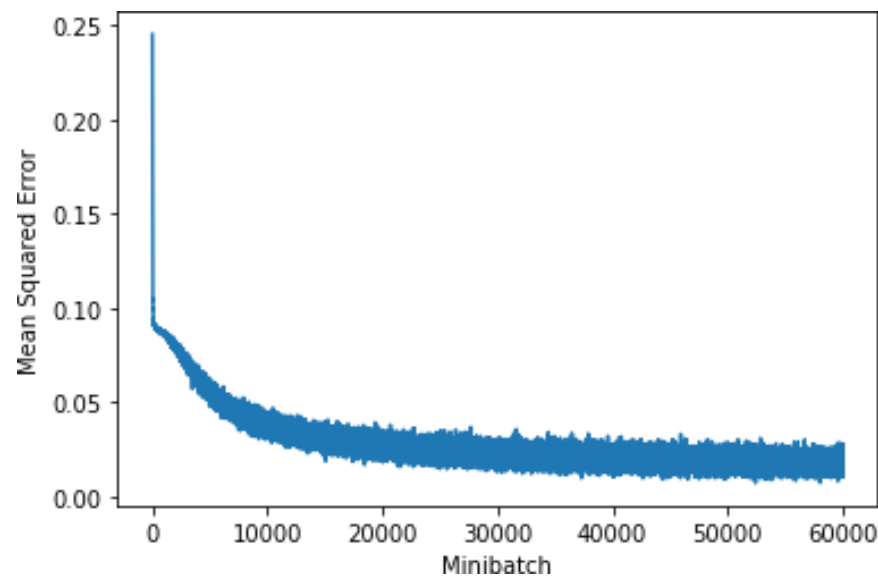
Implementing Multilayer Perceptrons in PyTorch

1. Multilayer Perceptron Architecture
2. Nonlinear Activation Functions
- 3. Multilayer Perceptron Code Examples**
4. Overfitting and Underfitting
5. Cats & Dogs and Custom Data Loaders

Multilayer Perceptron with Sigmoid Activation and MSE Loss

VS

Multilayer Perceptron with Softmax Activation and Cross Entropy Loss



Dead Neurons

- ReLU is probably the most popular activation function (simple to compute, fast, good results)
- But esp. ReLU neurons might "die" during training
- Can happen if, e.g., input is so large/small that net input is so small that ReLUs never recover (gradient 0 at $x < 0$)
- Not necessarily bad, can be considered as a form of regularization
- (compared to sigmoid/Tanh, ReLU suffers less from vanishing gradient problem but can more easily "explode")

Wide vs Deep Architectures (Breadth vs Depth)

MLP's with one (large) hidden unit are universal function approximators [1-3] already why do we want to use deeper architectures?

1 Balázs Csanád Csáji (2001) Approximation with Artificial Neural Networks; Faculty of Sciences; Eötvös Loránd University, Hungary

2 Cybenko, G. (1989) "Approximations by superpositions of sigmoidal functions", Mathematics of Control, Signals, and Systems, 2(4), 303–314. doi:10.1007/BF02551274

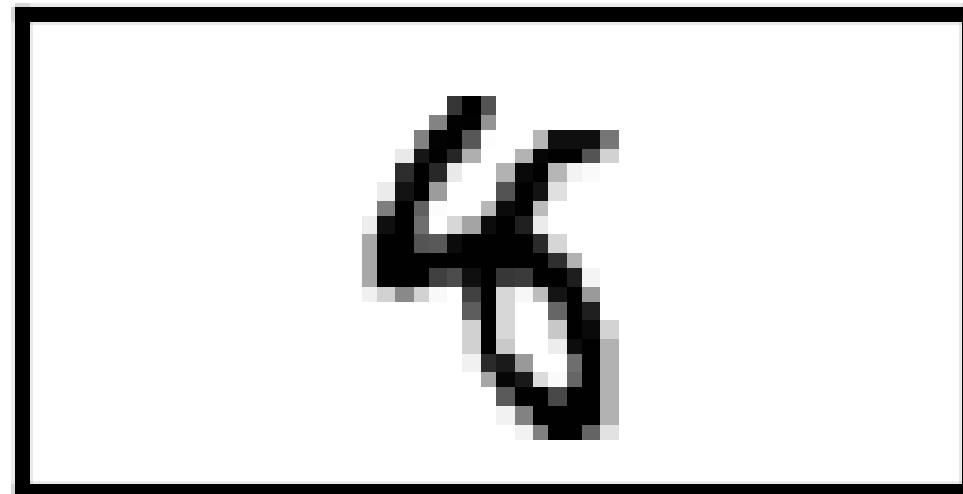
3 Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. Neural networks, 2(5), 359-366.

Wide vs Deep Architectures (Breadth vs Depth)

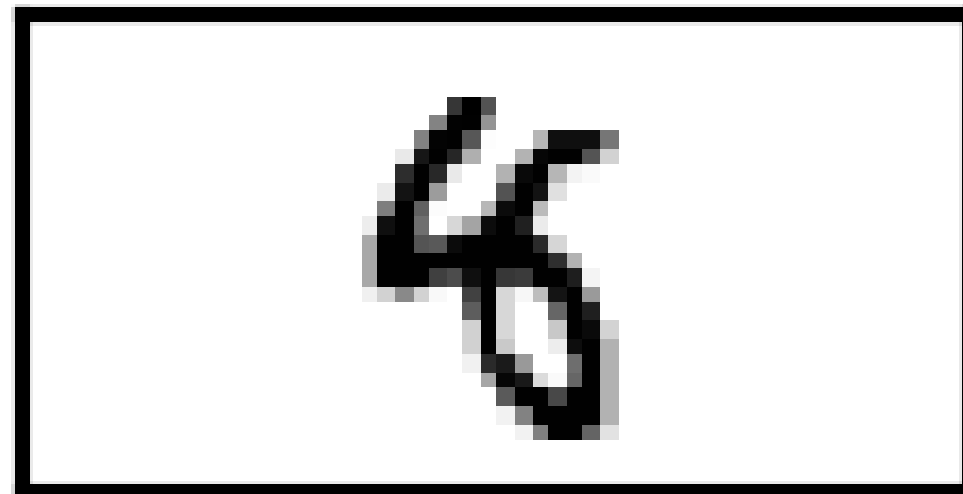
- Can achieve the same expressiveness with more layers but fewer parameters (combinatorics);
fewer parameters => less overfitting
- Also, having more layers provides some form of regularization: later layers are constrained on the behavior of earlier layers
- However, more layers => vanishing/exploding gradients
- Later: different layers for different levels of feature abstraction (DL is really more about feature learning than just stacking multiple layers)

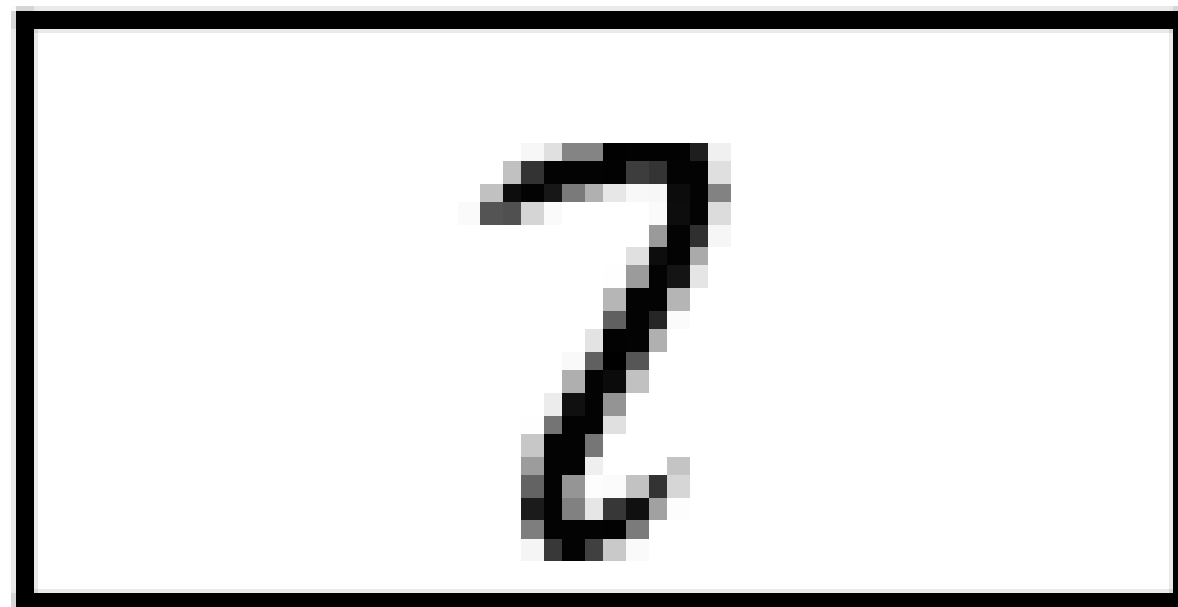
The problems with models that are too simple and models that fit the training data "too well"

1. Multilayer Perceptron Architecture
2. Nonlinear Activation Functions
3. Multilayer Perceptron Code Examples
- 4. Overfitting and Underfitting**
5. Cats & Dogs and Custom Data Loaders

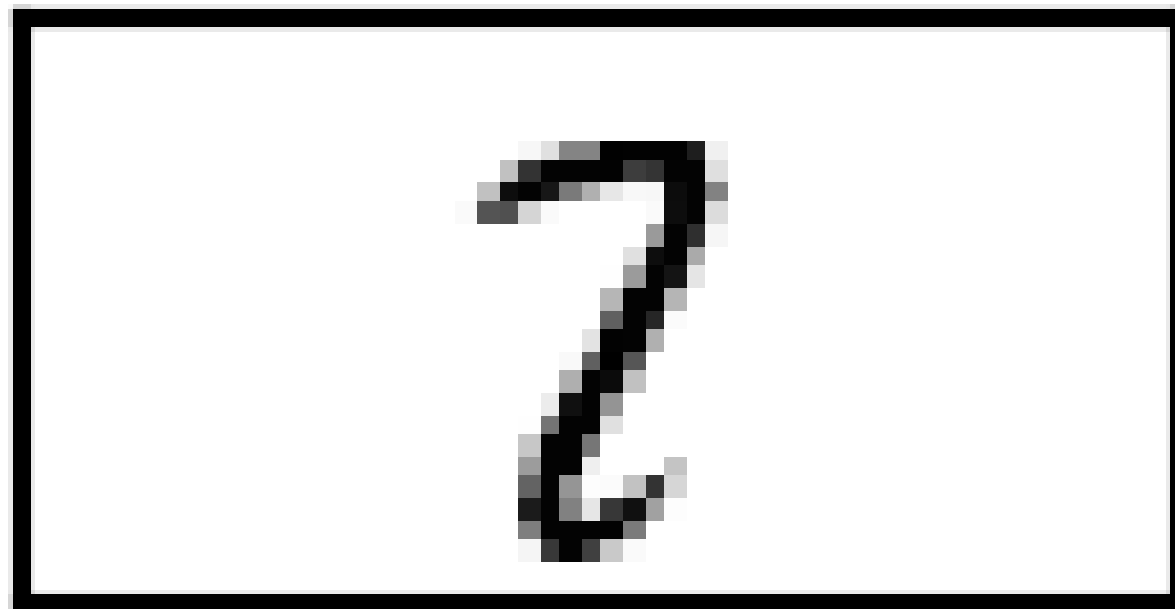


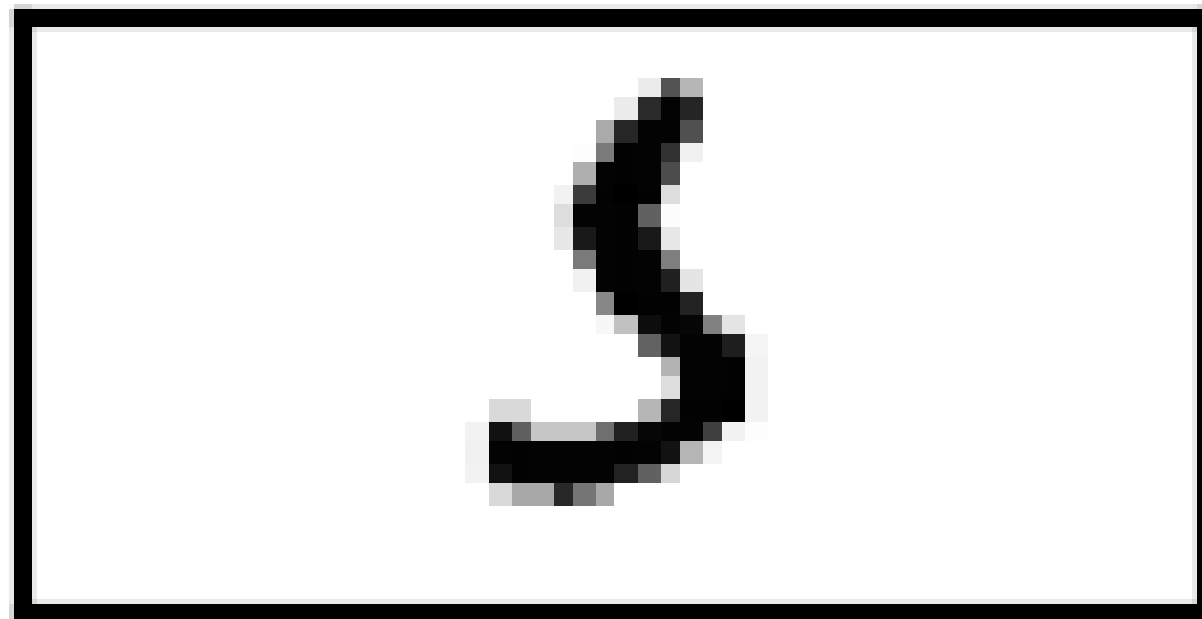
6) t: 8 p: 4



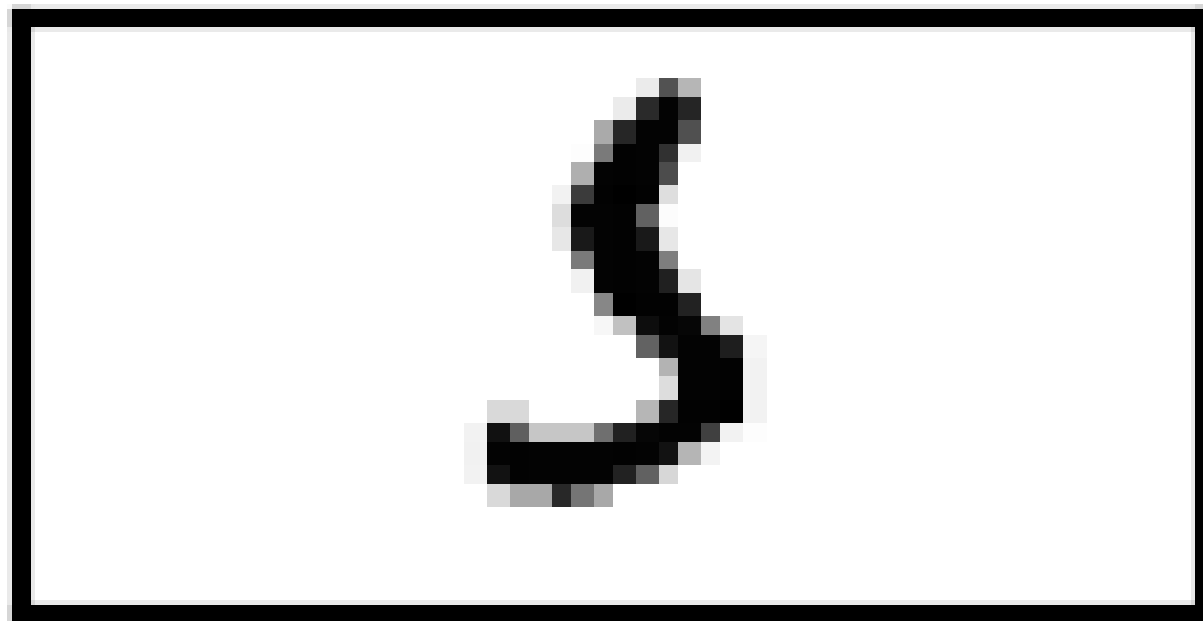


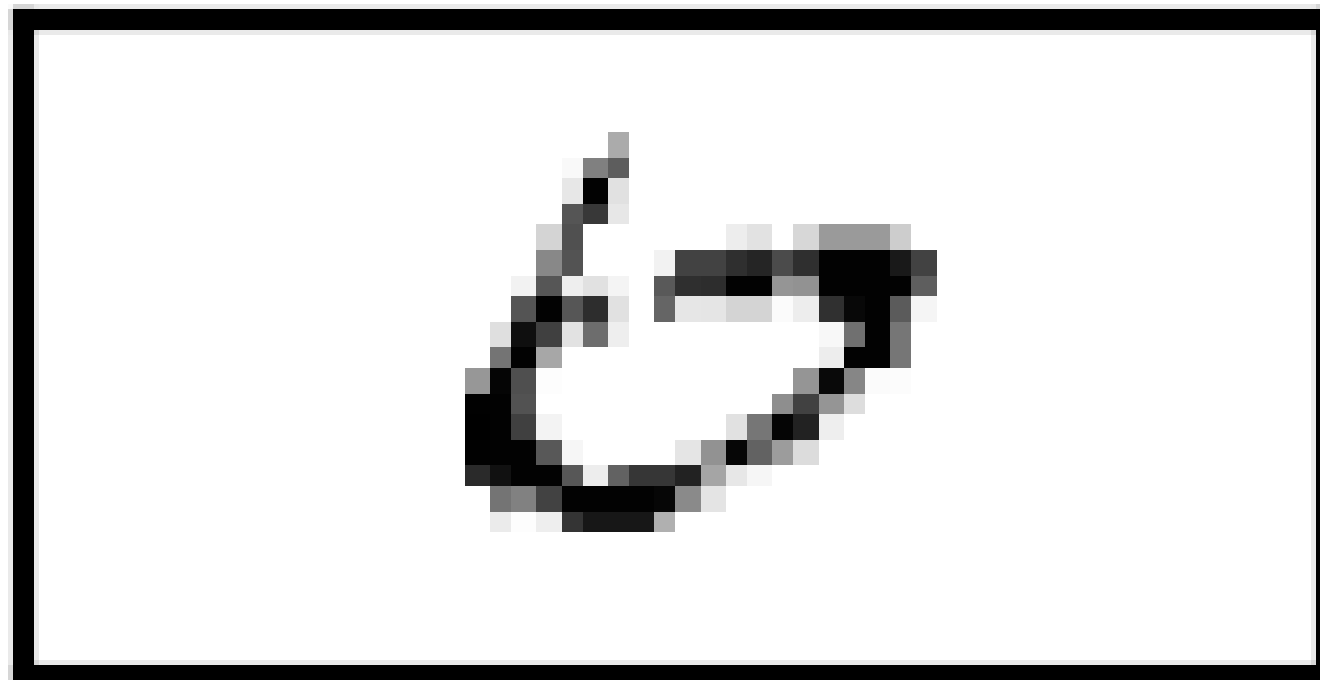
8) t: 2 p: 7



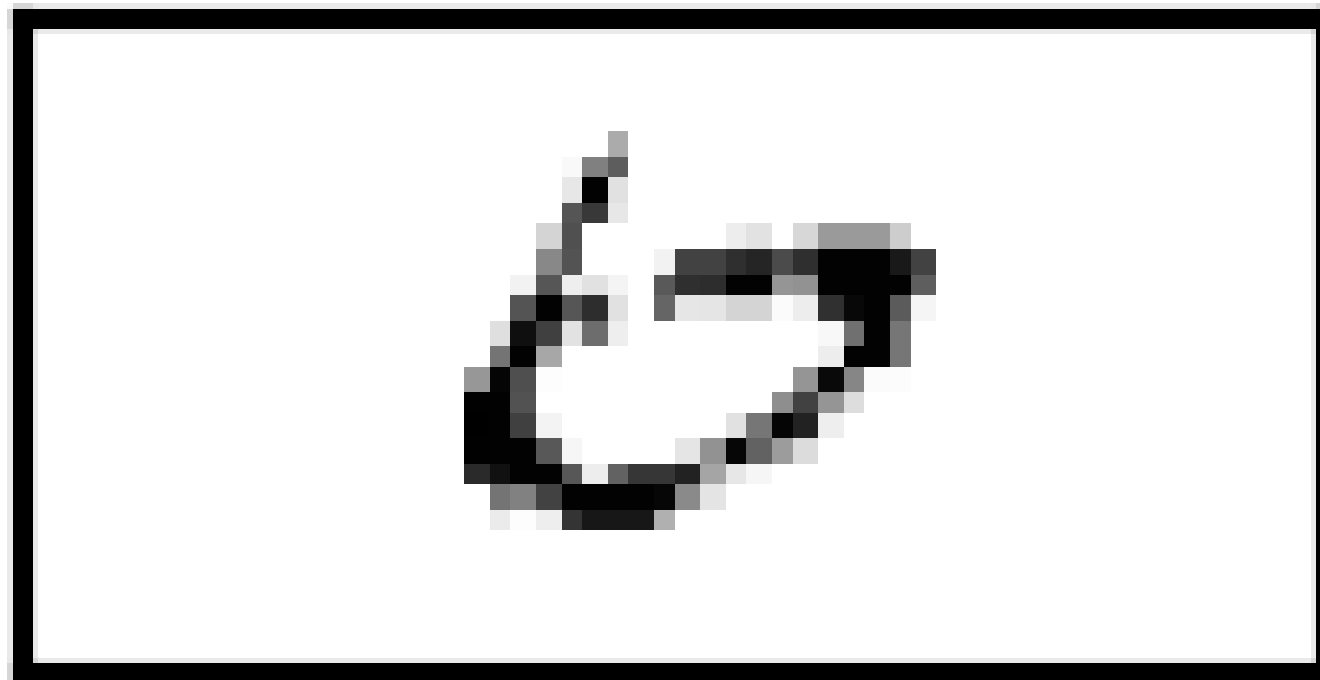


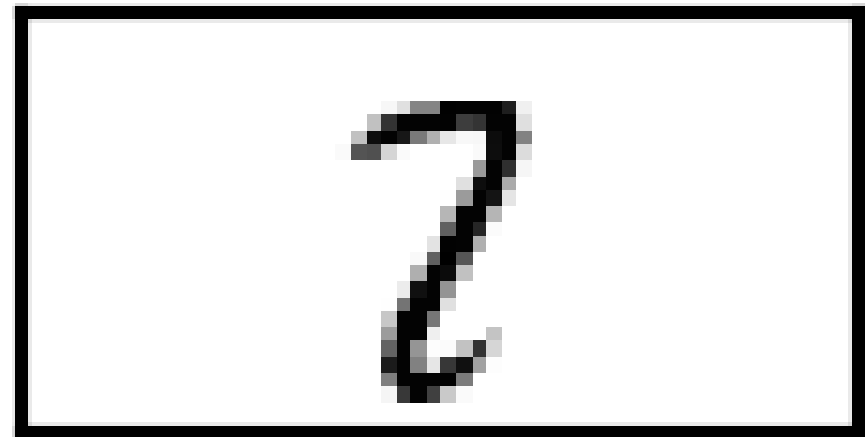
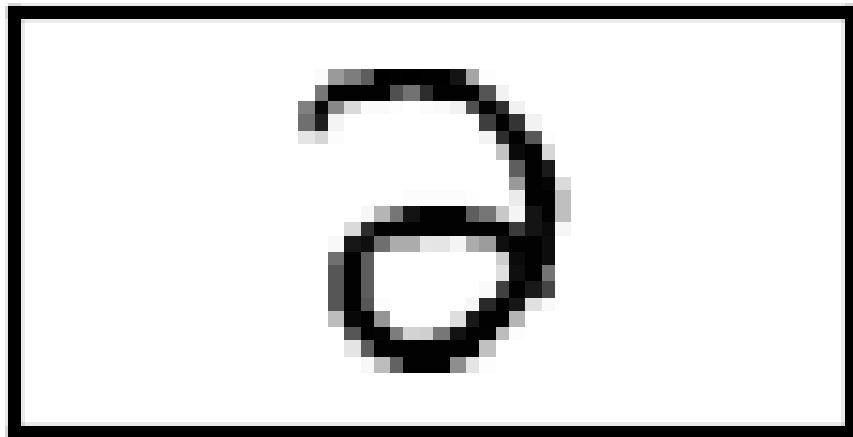
10) t: 5 p: 3



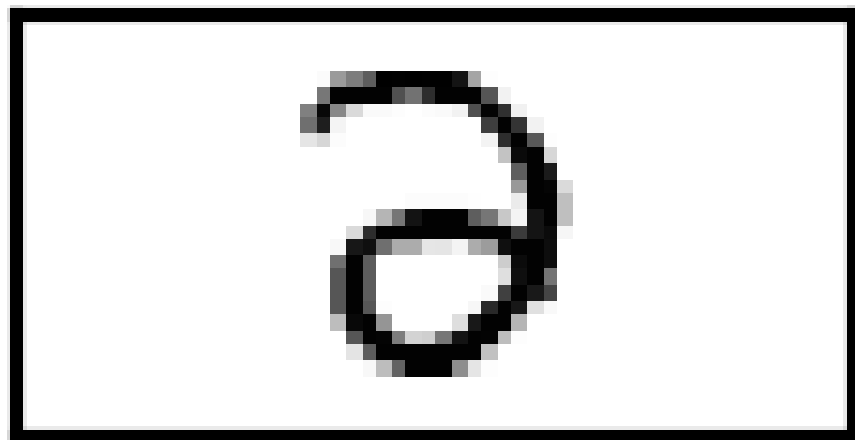


15) t: 6 p: 0

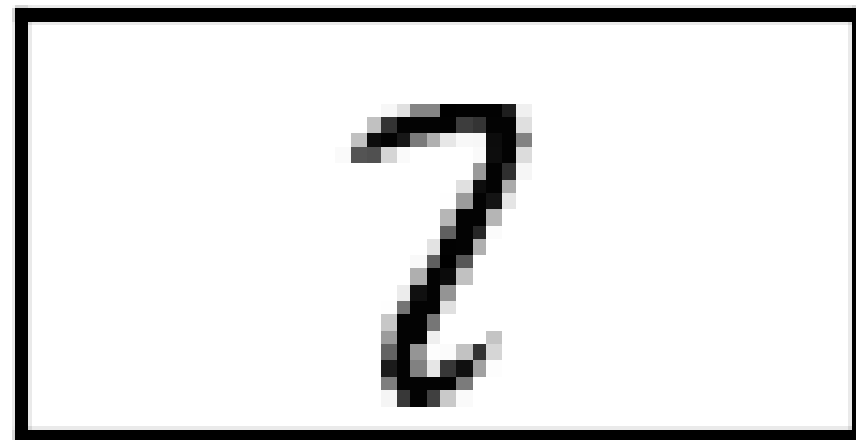




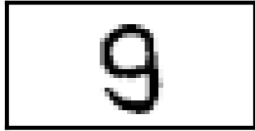
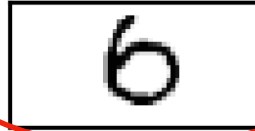
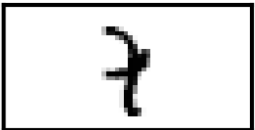
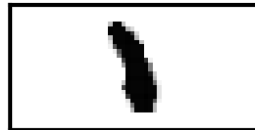
7) t: 2 p: 0



8) t: 2 p: 7

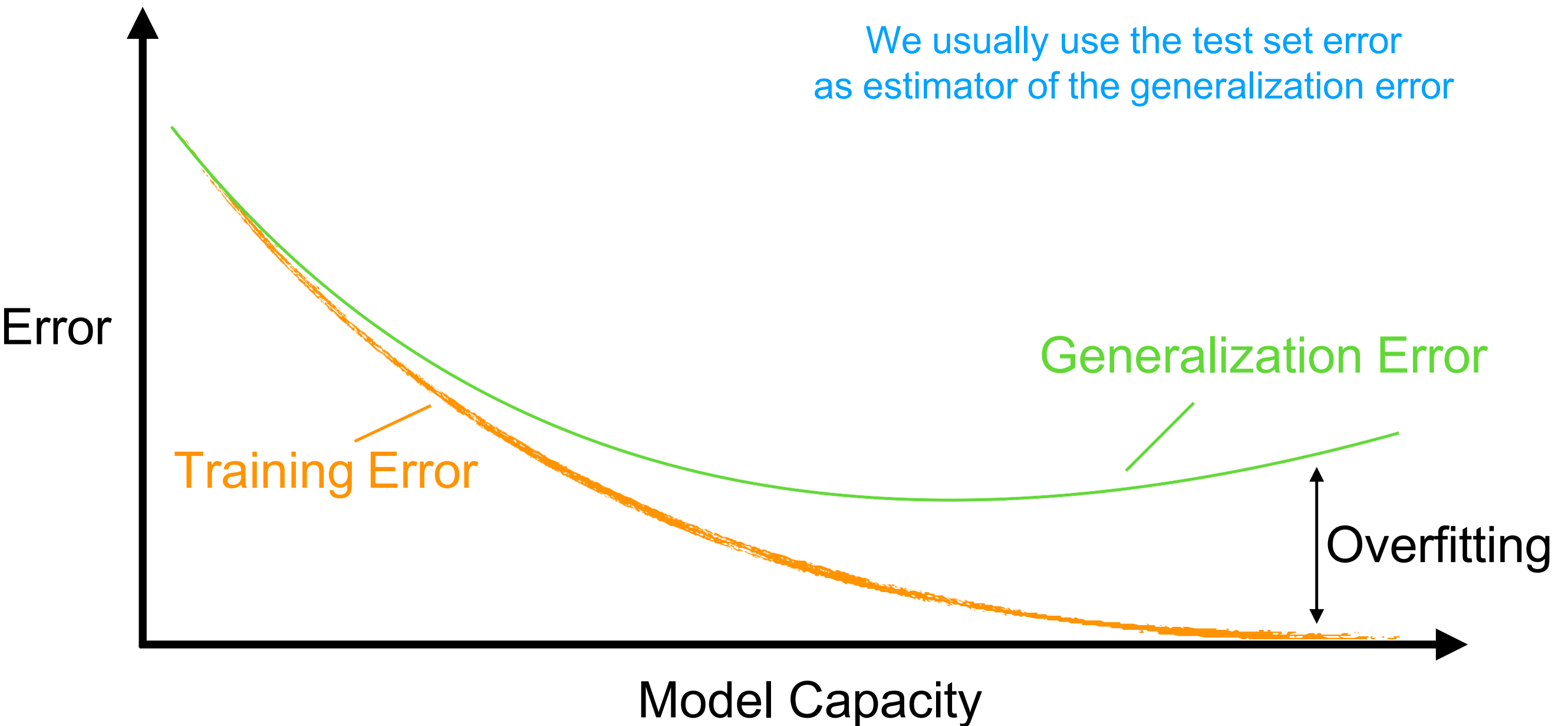


Recommended Practice: Looking at Some Failure Cases

1) t: 4 p: 0 	2) t: 2 p: 4 	3) t: 9 p: 3 	4) t: 4 p: 6 	5) t: 6 p: 0 
6) t: 8 p: 4 	7) t: 2 p: 0 	8) t: 2 p: 7 	9) t: 4 p: 9 	10) t: 5 p: 3 
11) t: 3 p: 7 	12) t: 8 p: 2 	13) t: 4 p: 6 	14) t: 8 p: 7 	15) t: 6 p: 0 
16) t: 9 p: 8 	17) t: 9 p: 3 	18) t: 8 p: 2 	19) t: 5 p: 3 	20) t: 9 p: 4 
21) t: 7 p: 3 	22) t: 4 p: 9 	23) t: 3 p: 7 	24) t: 4 p: 6 	25) t: 1 p: 8 

Failure cases of a ~93% accuracy (not very good, but beside the point)
2-layer (1-hidden layer) MLP on MNIST
(where t =target class and p =predicted class)

Overfitting and Underfitting



Bias-Variance Decomposition

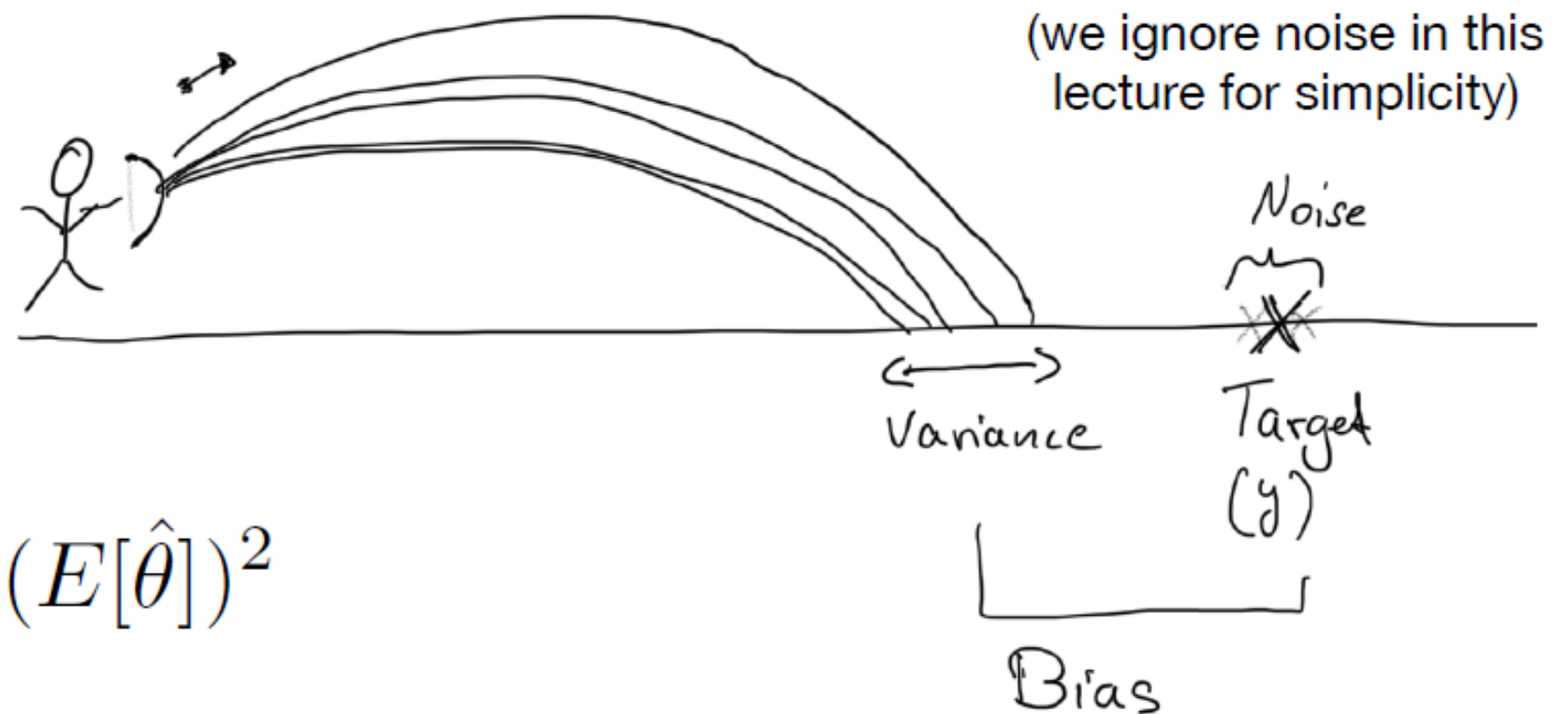
General Definition:

Intuition:

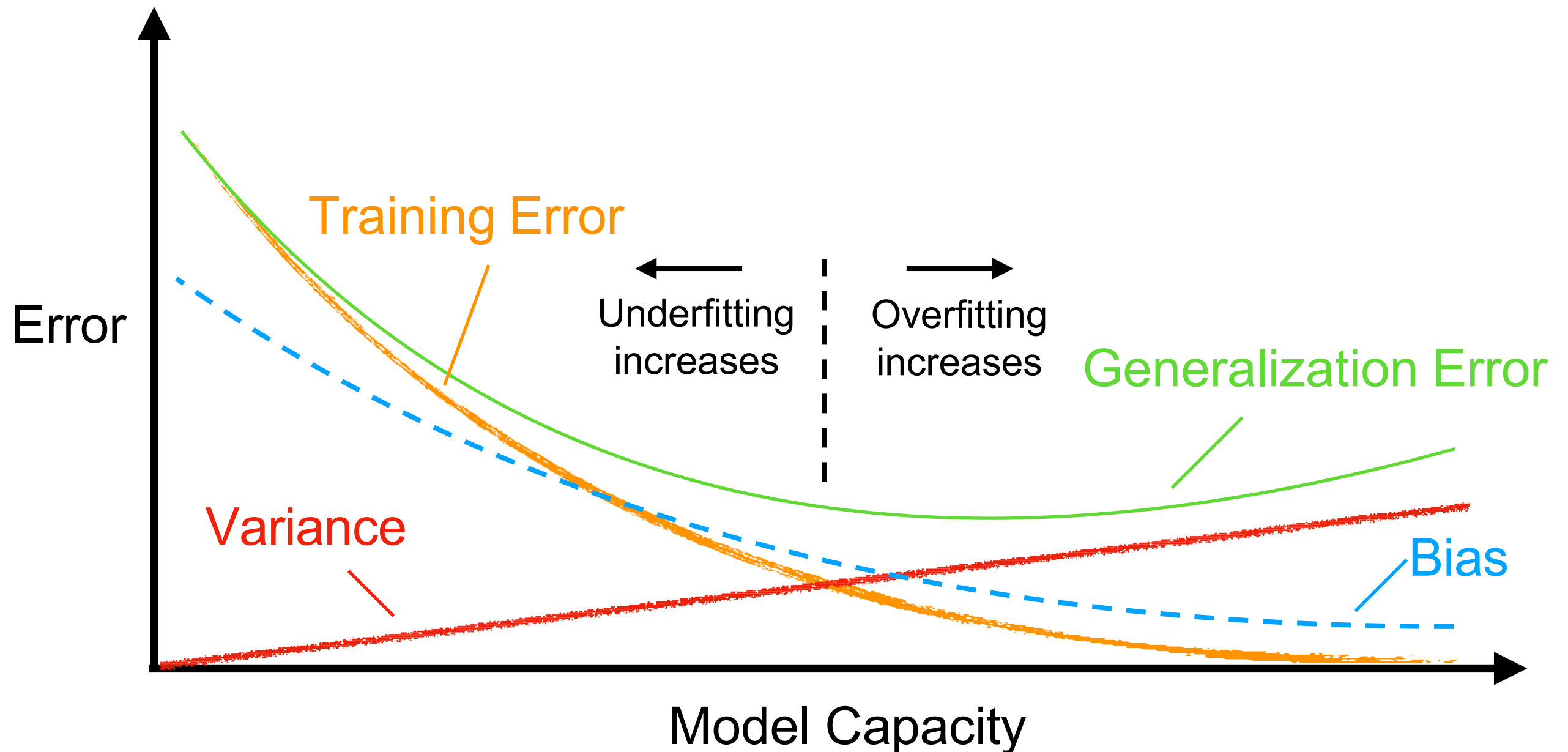
$$\text{Bias}_\theta[\hat{\theta}] = E[\hat{\theta}] - \theta$$

$$\text{Var}_\theta[\hat{\theta}] = E[\hat{\theta}^2] - (E[\hat{\theta}])^2$$

$$\text{Var}_\theta[\hat{\theta}] = E[(E[\hat{\theta}] - \hat{\theta})^2]$$



Bias & Variance vs Overfitting & Underfitting



capacity: abstract concept meaning roughly the number of parameters of the model times how efficiently the parameters are used

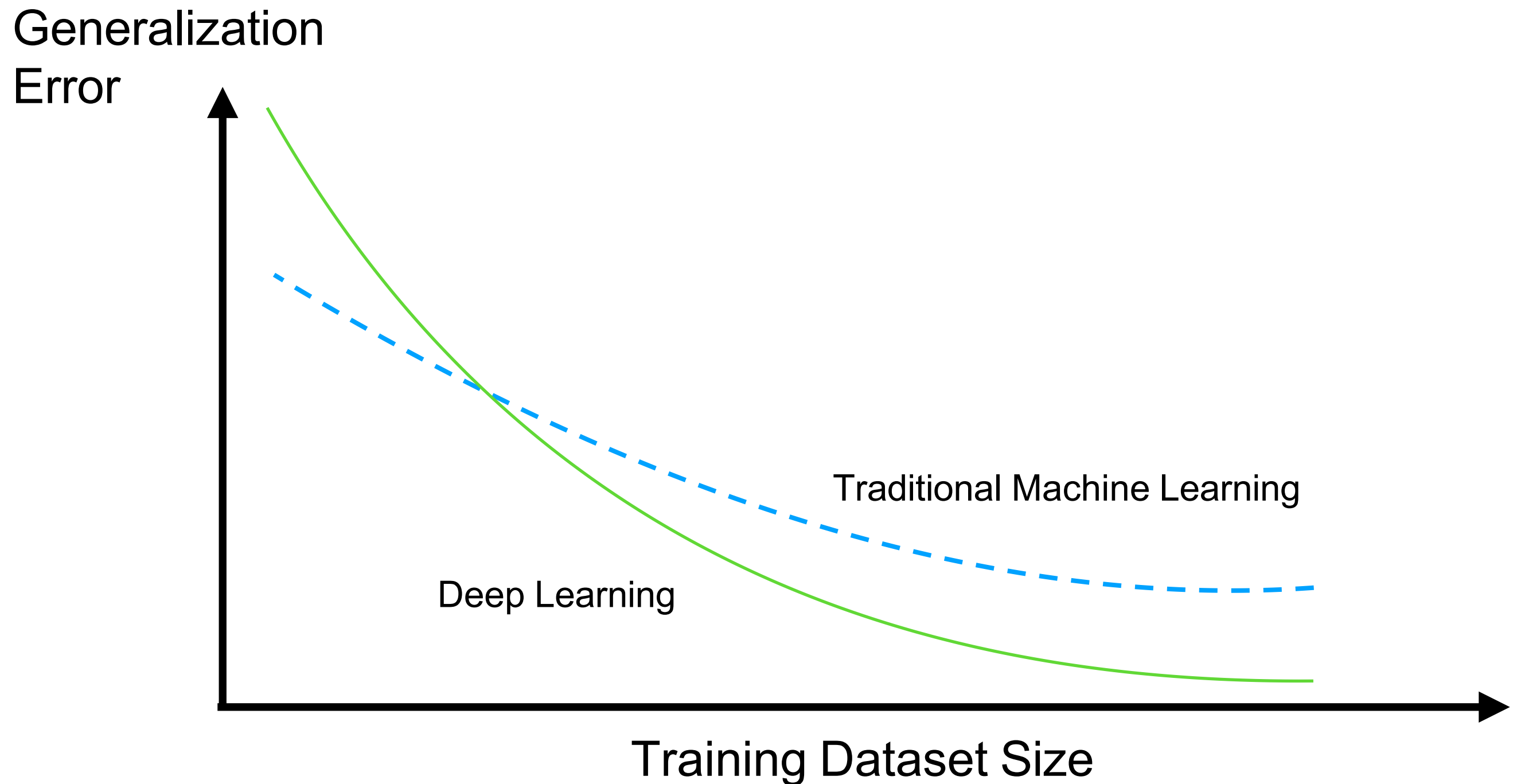
Further Reading Material

- A more detailed understanding of bias and variance is not mandatory for this class
- Also, train/valid/test splits are usually sufficient for training & estimating the generalization performance in deep learning
- However, if you are interested, refer to the paper below:

Raschka, S. (2018). Model evaluation, model selection, and algorithm selection in machine learning. *arXiv preprint arXiv:1811.12808*.

<https://arxiv.org/pdf/1811.12808.pdf>

Deep Learning Works Best with Large Datasets



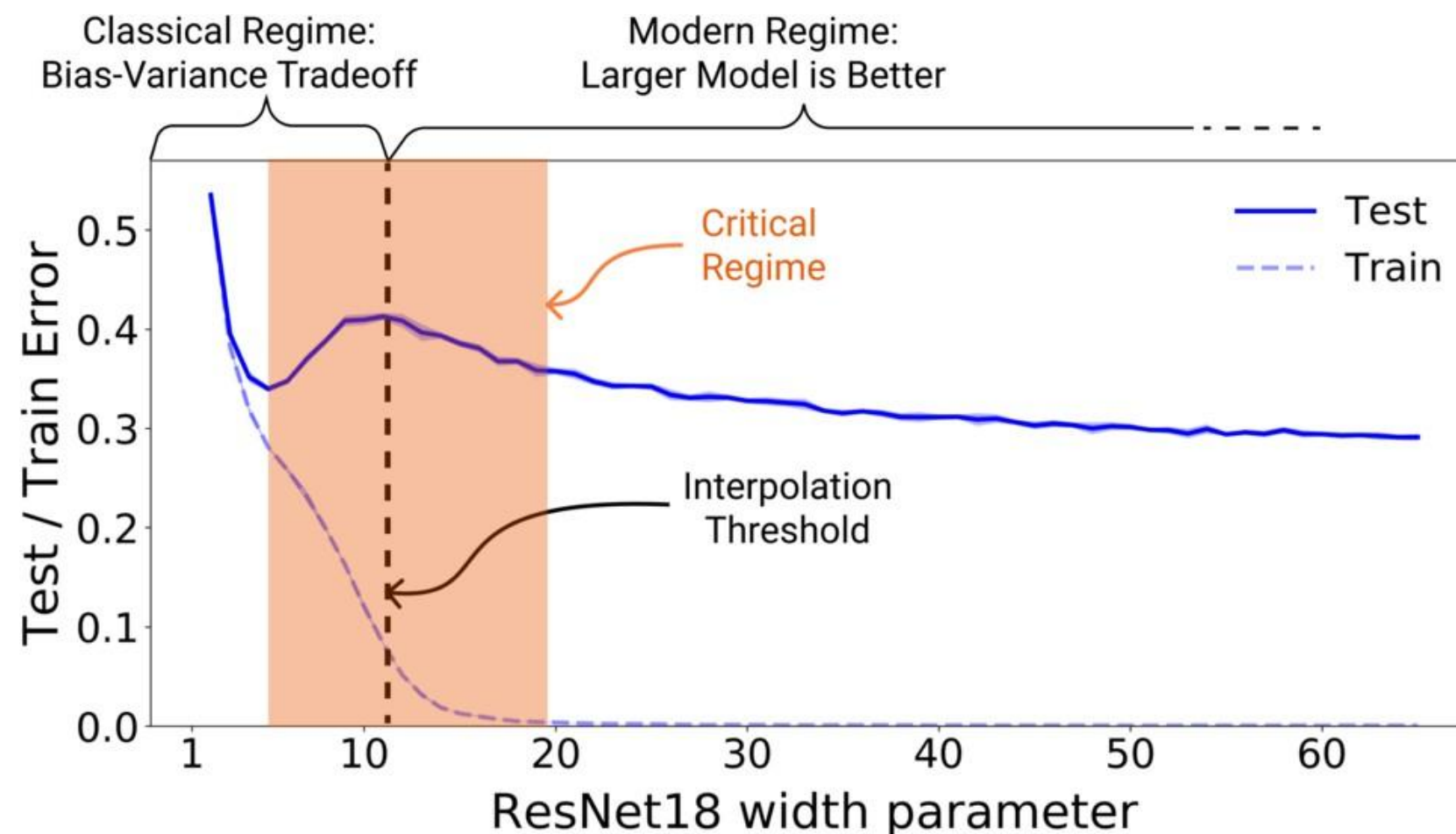
[Submitted on 4 Dec 2019]

Deep Double Descent: Where Bigger Models and More Data Hurt

Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, Ilya Sutskever

<https://arxiv.org/abs/1912.02292>

Architectures: CNNs (standard & ResNet) and transformers trained with cross-entropy loss

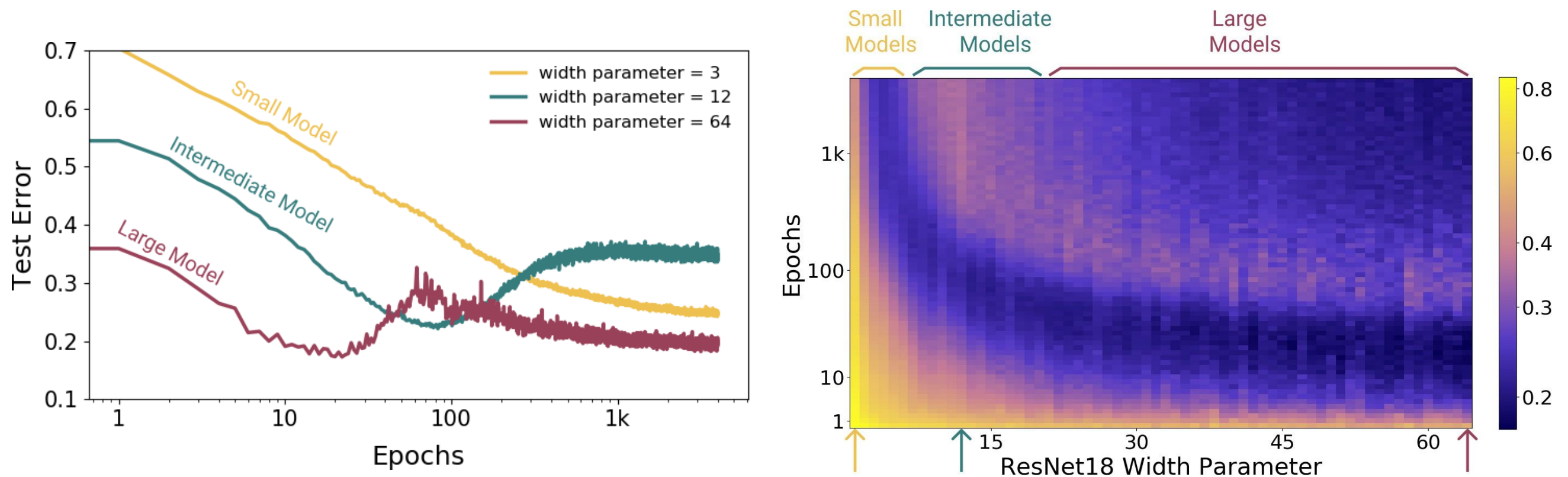


[Submitted on 4 Dec 2019]

Deep Double Descent: Where Bigger Models and More Data Hurt

Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, Ilya Sutskever

<https://arxiv.org/abs/1912.02292>



Thoughts:

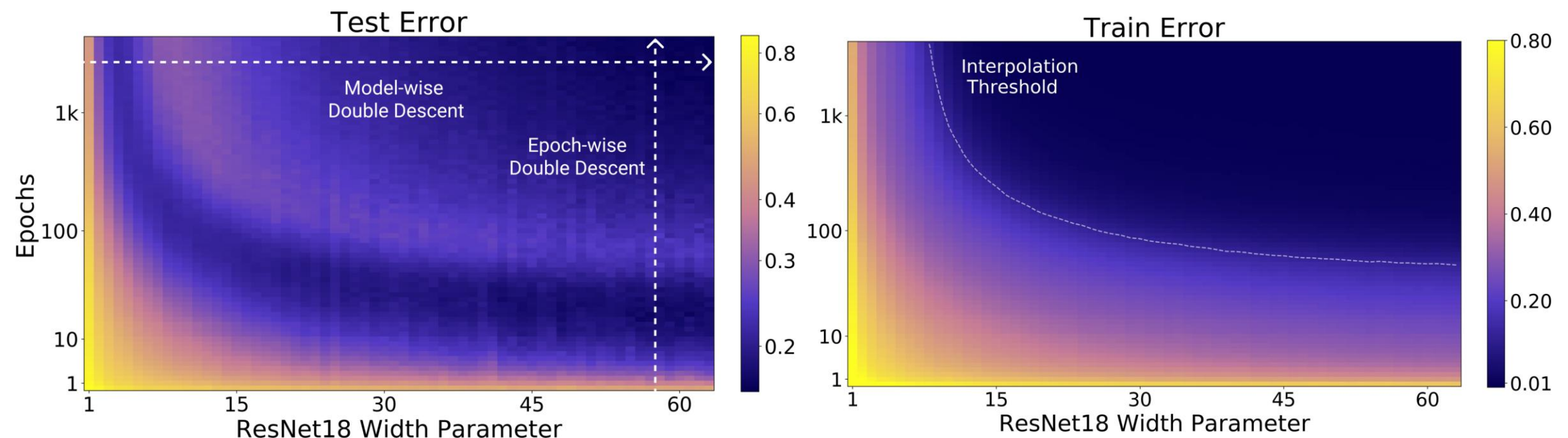
- at critical region, only one model fits the data well and is very sensitive to noise
- overparametrized models: many fit the data well, SGD finds one that memorizes the training set but also performs well on the test set

[Submitted on 4 Dec 2019]

Deep Double Descent: Where Bigger Models and More Data Hurt

Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, Ilya Sutskever

<https://arxiv.org/abs/1912.02292>

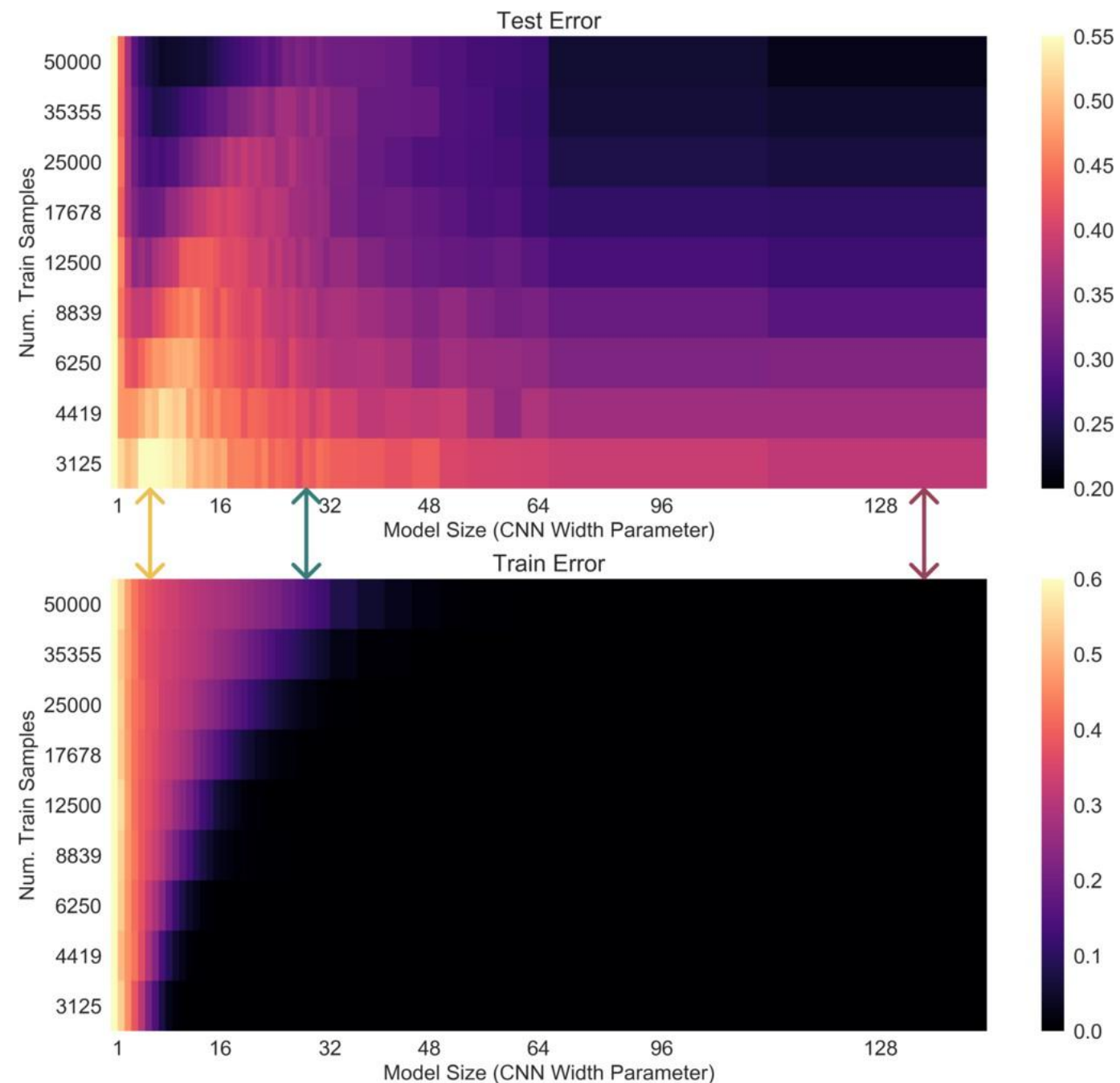


[Submitted on 4 Dec 2019]

Deep Double Descent: Where Bigger Models and More Data Hurt

Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, Ilya Sutskever

<https://arxiv.org/abs/1912.02292>

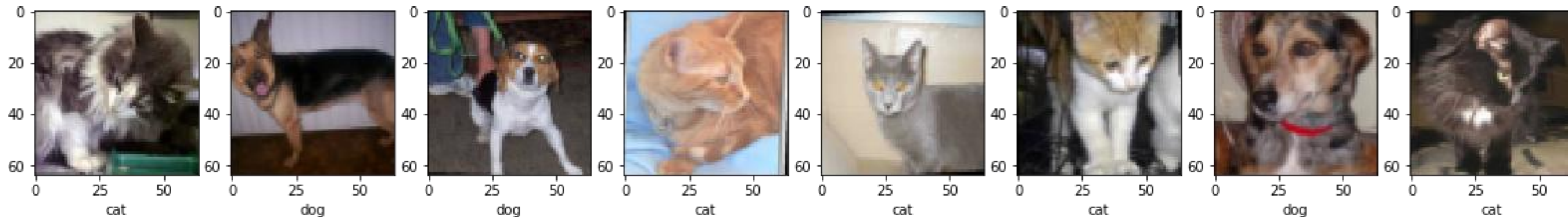
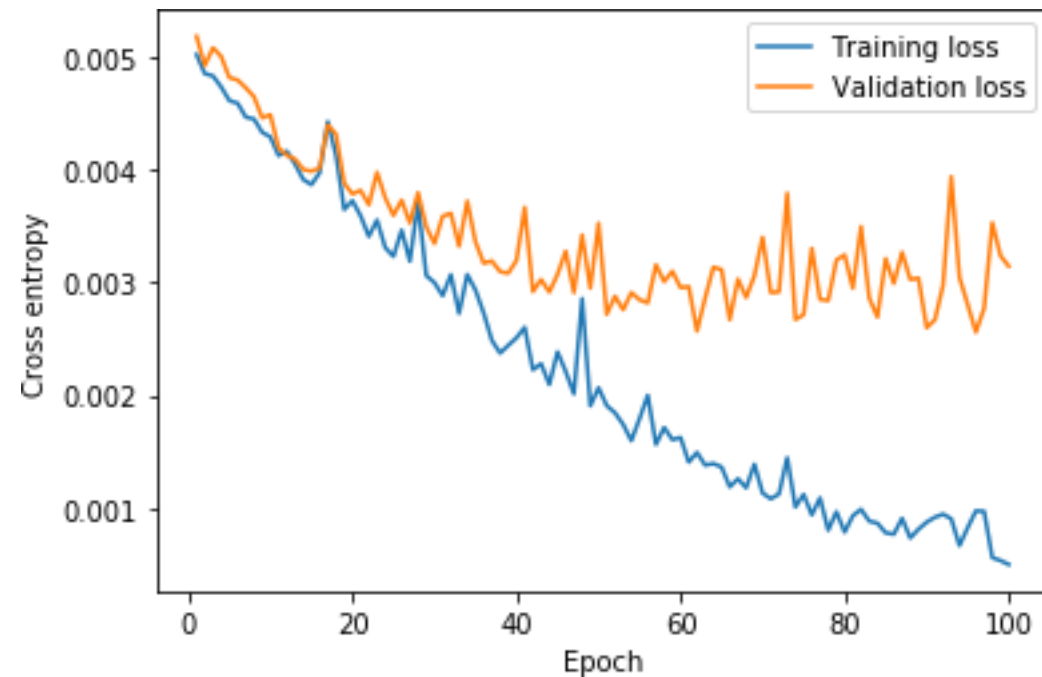


Training multilayer perceptrons with your own datasets

1. Multilayer Perceptron Architecture
2. Nonlinear Activation Functions
3. Multilayer Perceptron Code Examples
4. Overfitting and Underfitting
- 5. Cats & Dogs and Custom Data Loaders**

VGG16 Convolutional Neural Network for Kaggle's Cats and Dogs Images

A "real world" example



```
model.eval()  
with torch.set_grad_enabled(False): # save memory during inference  
    test_acc, test_loss = compute_accuracy_and_loss(model, test_loader, DEVICE)  
    print(f'Test accuracy: {test_acc:.2f}%')
```

Test accuracy: 88.28%

https://github.com/rasbt/deeplearning-models/blob/master/pytorch_ipynb/cnn/cnn-vgg16-cats-dogs.ipynb

Training/Validation/Test splits

Ratio depends on the dataset size, but a 80/5/15 split is usually a good idea

- Training set is used for training, it is not necessary to plot the training accuracy during training but it can be useful
- Validation set accuracy provides a rough estimate of the generalization performance (it can be optimistically biased if you design the network to do well on the validation set ("information leakage"))
- Test set should only be used once to get an unbiased estimate of the generalization performance

Training/Validation/Test splits

Epoch: 001/100 | Batch 000/156 | Cost: 1136.9125
Epoch: 001/100 | Batch 120/156 | Cost: 0.6327
Epoch: 001/100 Train Acc.: 63.35% | Validation Acc.: 62.12%
Time elapsed: 3.09 min
Epoch: 002/100 | Batch 000/156 | Cost: 0.6675
Epoch: 002/100 | Batch 120/156 | Cost: 0.6640
Epoch: 002/100 Train Acc.: 66.05% | Validation Acc.: 66.32%
Time elapsed: 6.15 min
Epoch: 003/100 | Batch 000/156 | Cost: 0.6137
Epoch: 003/100 | Batch 120/156 | Cost: 0.6311
Epoch: 003/100 Train Acc.: 65.82% | Validation Acc.: 63.76%
Time elapsed: 9.21 min
Epoch: 004/100 | Batch 000/156 | Cost: 0.5993
Epoch: 004/100 | Batch 120/156 | Cost: 0.5832
Epoch: 004/100 Train Acc.: 66.75% | Validation Acc.: 64.52%
Time elapsed: 12.27 min
Epoch: 005/100 | Batch 000/156 | Cost: 0.5918
Epoch: 005/100 | Batch 120/156 | Cost: 0.5747
Epoch: 005/100 Train Acc.: 68.29% | Validation Acc.: 67.00%
Time elapsed: 15.33 min
...

```
model.eval()  
with torch.set_grad_enabled(False): # save memory during inference  
    test_acc, test_loss = compute_accuracy_and_loss(model, test_loader, DEVICE)  
    print(f'Test accuracy: {test_acc:.2f}%')
```

Test accuracy: 88.28%

Parameters vs Hyperparameters

- weights (weight parameters)
- biases (bias units)
- minibatch size
- data normalization schemes
- number of epochs
- number of hidden layers
- number of hidden units
- learning rates
- (random seed, why?)
- loss function
- various weights (weighting terms)
- activation function types
- regularization schemes (more later)
- weight initialization schemes (more later)
- optimization algorithm type (more later)
- ...

(Mostly no scientific explanation, mostly engineering;
need to try many things -> "graduate student descent")

PyTorch DataLoader



https://twitter.com/_ScottCondon/status/1363494433715552259?s=20

Custom DataLoader Classes ...

- Example showing how you can create your own data loader to efficiently iterate through your own collection of images
(pretend the MNIST images there are some custom image collection)

<https://github.com/rasbt/stat453-deep-learning-ss20/blob/main/L09/code/custom-dataloader>

mnist_test
mnist_train
mnist_valid
custom-dataloader-example.ipynb
mnist_test.csv
mnist_train.csv
mnist_valid.csv

```
import torch
from PIL import Image
from torch.utils.data import Dataset
import os

class MyDataset(Dataset):

    def __init__(self, csv_path, img_dir, transform=None):

        df = pd.read_csv(csv_path)
        self.img_dir = img_dir
        self.img_names = df['File Name']
        self.y = df['Class Label']
        self.transform = transform

    def __getitem__(self, index):

        img = Image.open(os.path.join(self.img_dir,
                                       self.img_names[index]))

        if self.transform is not None:
            img = self.transform(img)

        label = self.y[index]
        return img, label

    def __len__(self):
        return self.y.shape[0]
```

Good news: We can solve non-linear problems now! Yay! :)

Bad news: Our multilayer neural nets have a lot of parameters now, and it's easy to overfit the the data! :(

Regularization

In the context of deep learning, regularization can be understood as the process of adding information / changing the objective function to prevent overfitting

Goal: Reduce Overfitting

usually achieved by reducing model capacity and/or reduction of the variance of the predictions (as explained last lecture)

Regularization / Regularizing Effects

Goal: reduce overfitting

usually achieved by reducing model capacity and/or reduction of the variance of the predictions (as explained last lecture)

Common Regularization Techniques for DNNs:

- Early stopping
- L_1/L_2 regularization (norm penalties)
- Dropout

Regularization (mathematics)

From Wikipedia, the free encyclopedia



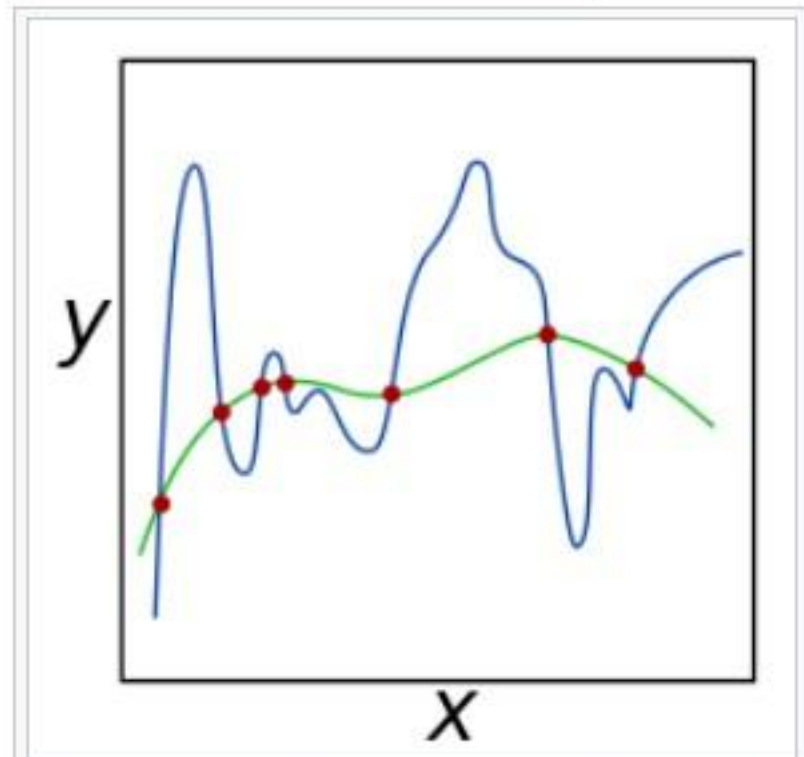
This article **only describes one highly specialized aspect of its associated subject**. Please help [improve this article](#) by adding more general information. The [talk page](#) may contain suggestions. *(November 2020)*

In [mathematics](#), [statistics](#), [finance](#),^[1] [computer science](#), particularly in [machine learning](#) and [inverse problems](#), **regularization** is the process of adding information in order to solve an [ill-posed problem](#) or to prevent [overfitting](#).^[2]

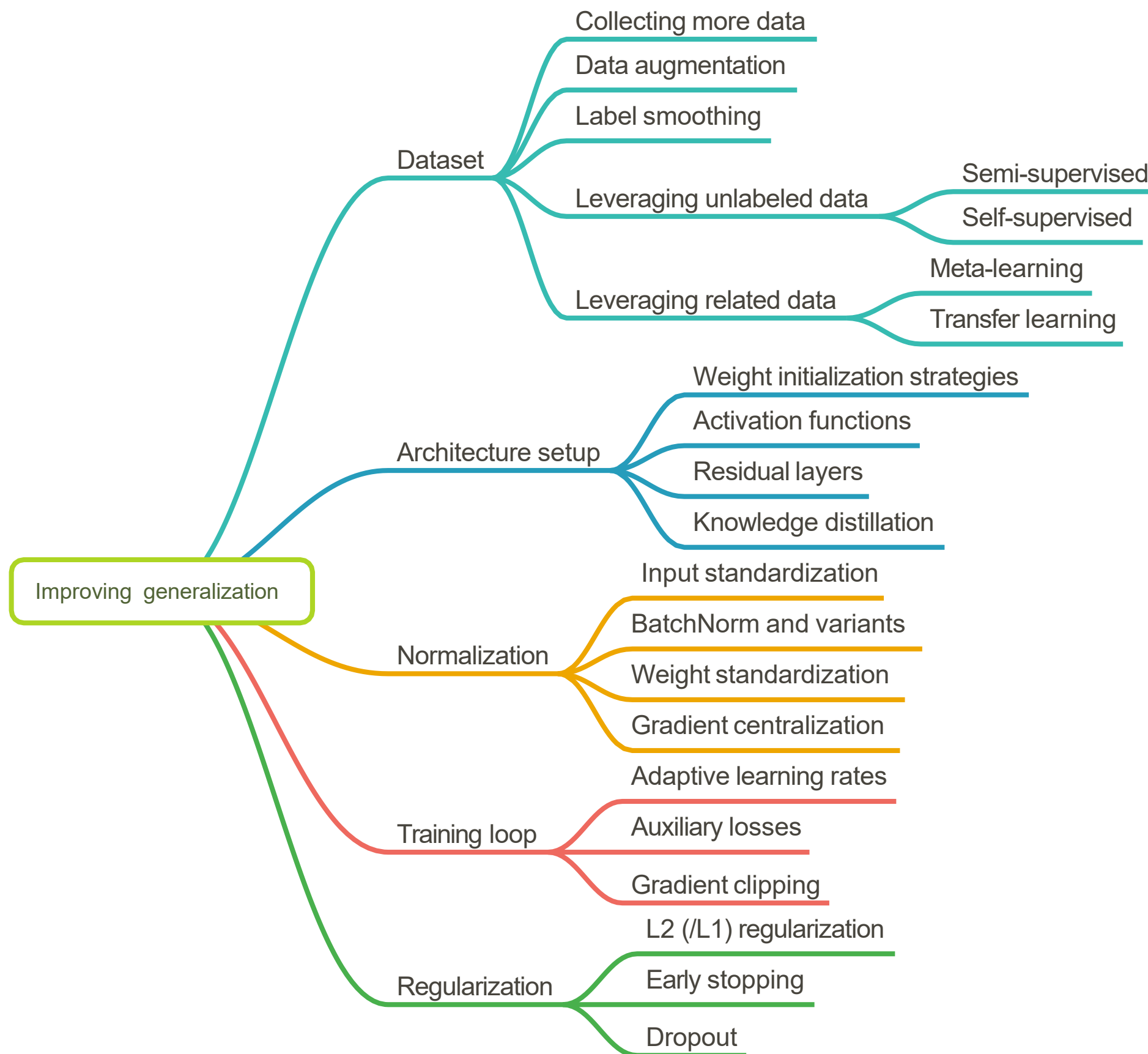
Regularization applies to objective functions in ill-posed optimization problems. The regularization term, or penalty, imposes a cost on the optimization function for overfitting the function or to find an optimal solution.

In [machine learning](#), [regularization is any modification](#) one makes to a learning algorithm that is intended to reduce its generalization error but not its training error^[3]

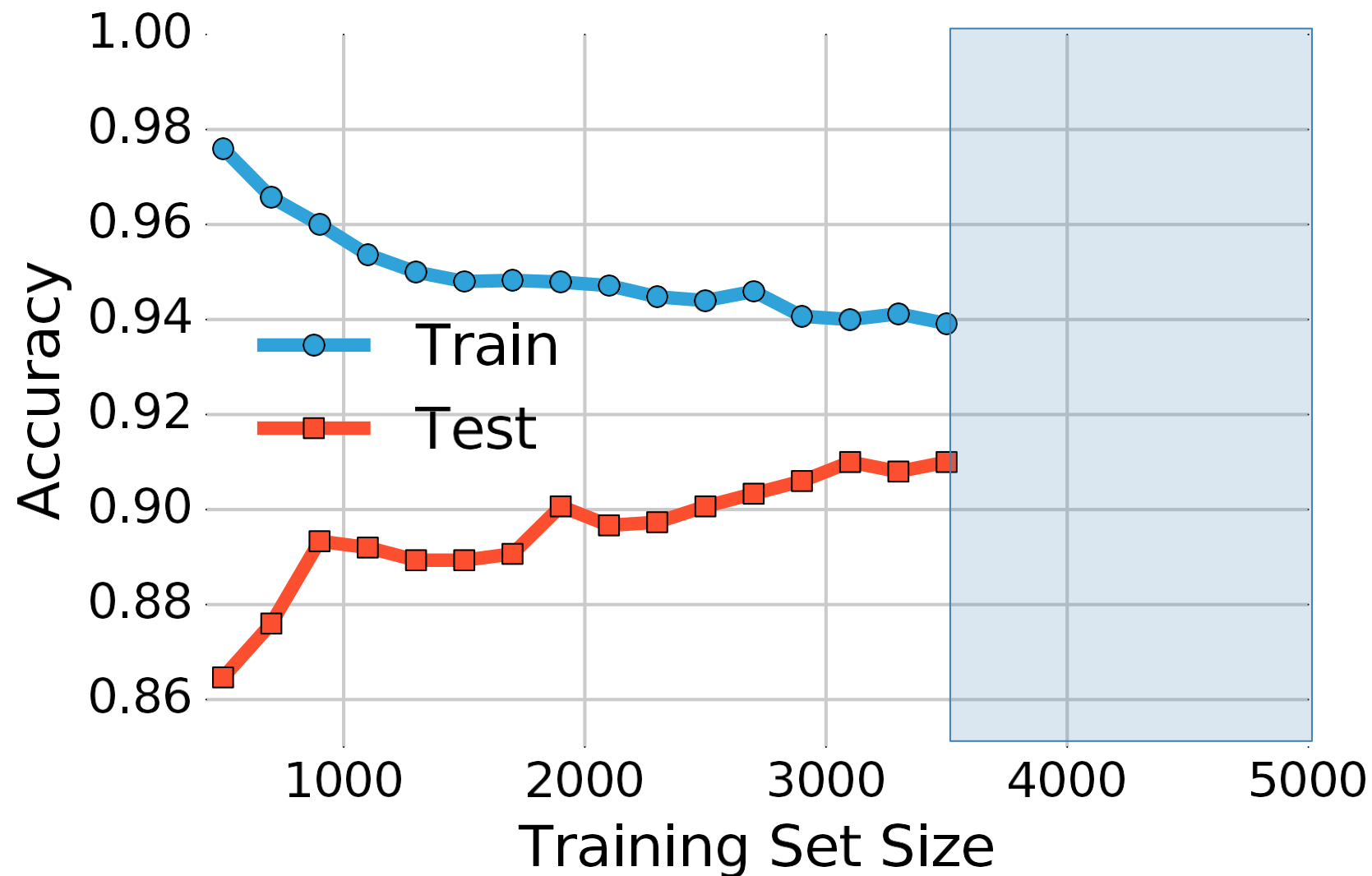
[Contents](#) [\[hide\]](#)



The green and blue functions both incur zero loss on the given data points. A learned



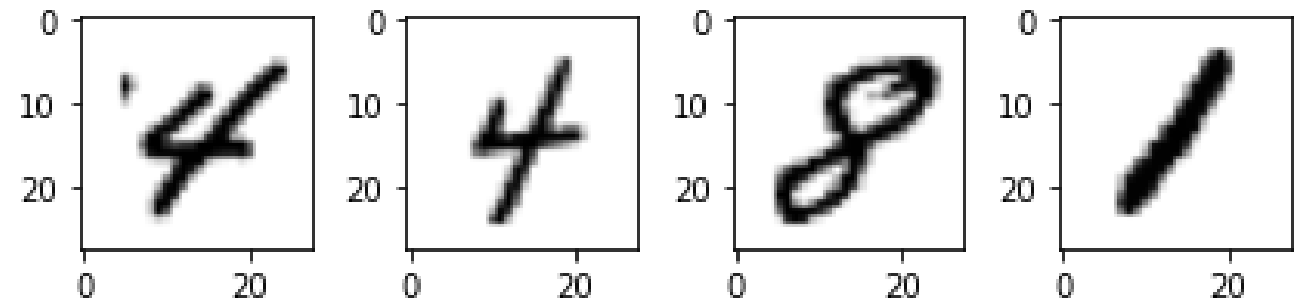
Often, the Best Way to Reduce Overfitting is Collecting More Data



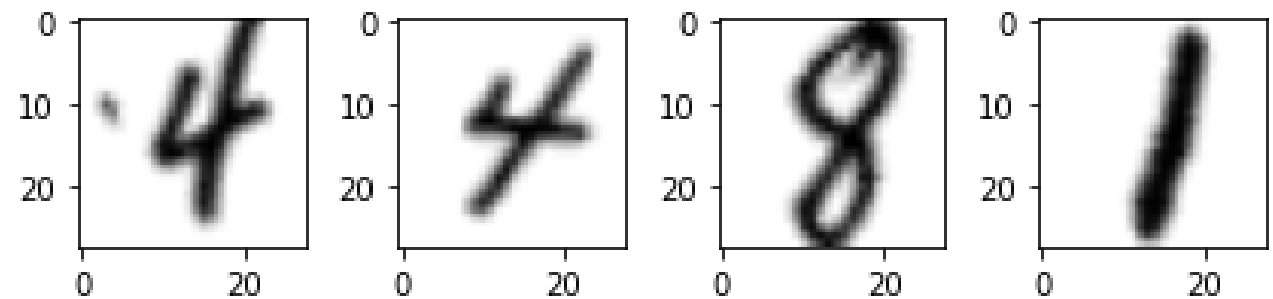
Softmax on MNIST subset (test set size is kept constant)

Data Augmentation in PyTorch via TorchVision

Original



Randomly Augmented



Early Stopping

=> Reducing Network's Capacity by Other Means

Step 1: Split your dataset into 3 parts (always recommended)

- use test set only once at the end (for unbiased estimate of generalization performance)
- use validation accuracy for tuning (always recommended)

Dataset

Training
dataset

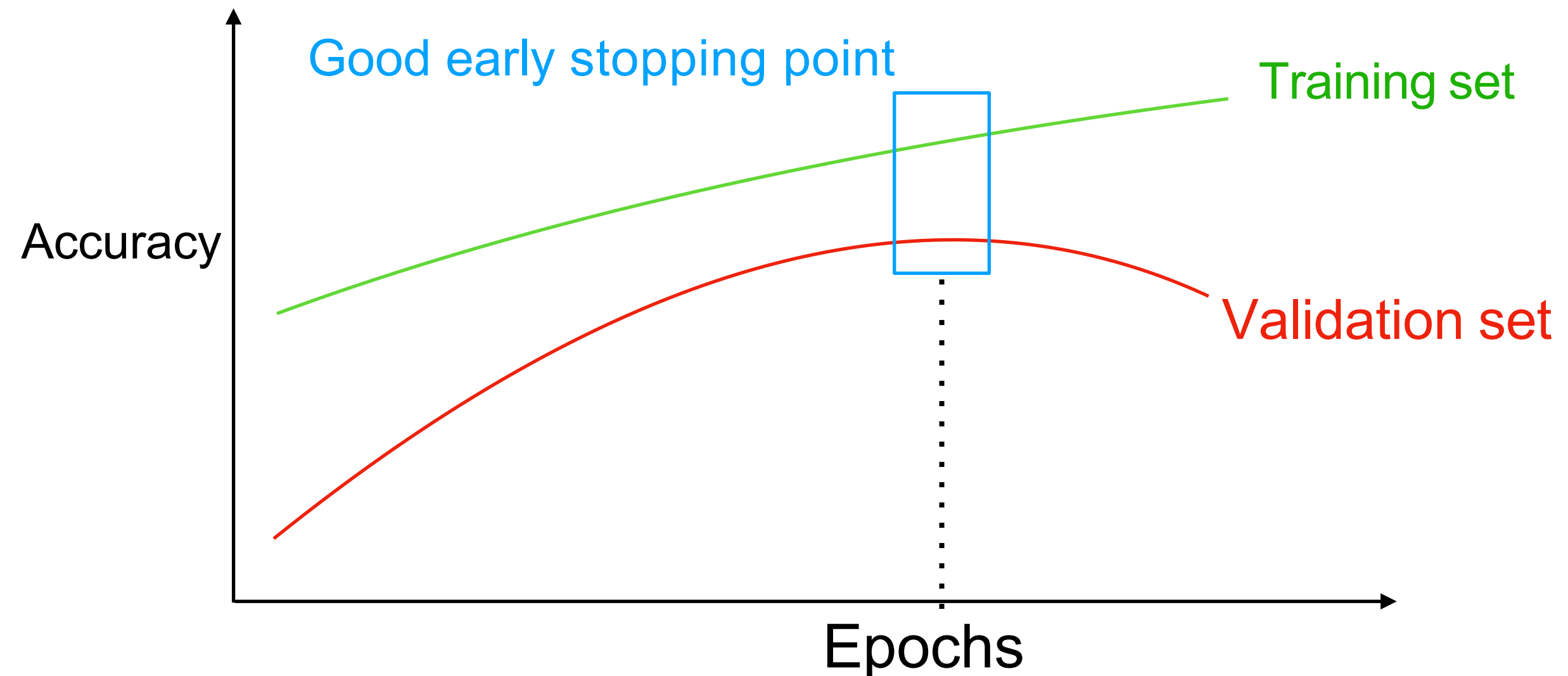
Validation
dataset

Test
dataset

Early Stopping

Step 2: Early stopping (not very common anymore)

- reduce overfitting by observing the training/validation accuracy gap during training and then stop at the "right" point



L_1/L_2 Regularization

Adding a Penalty Against Complexity

As I am sure you already know it from various statistics classes, we will keep it short:

- L_1 -regularization $\Rightarrow$ LASSO regression
- L_2 -regularization $\Rightarrow$ Ridge regression (Thikonov regularization)

Basically, a "weight shrinkage" or a "penalty against complexity"

L₂ Regularization for Linear Models (e.g., Logistic Regression)

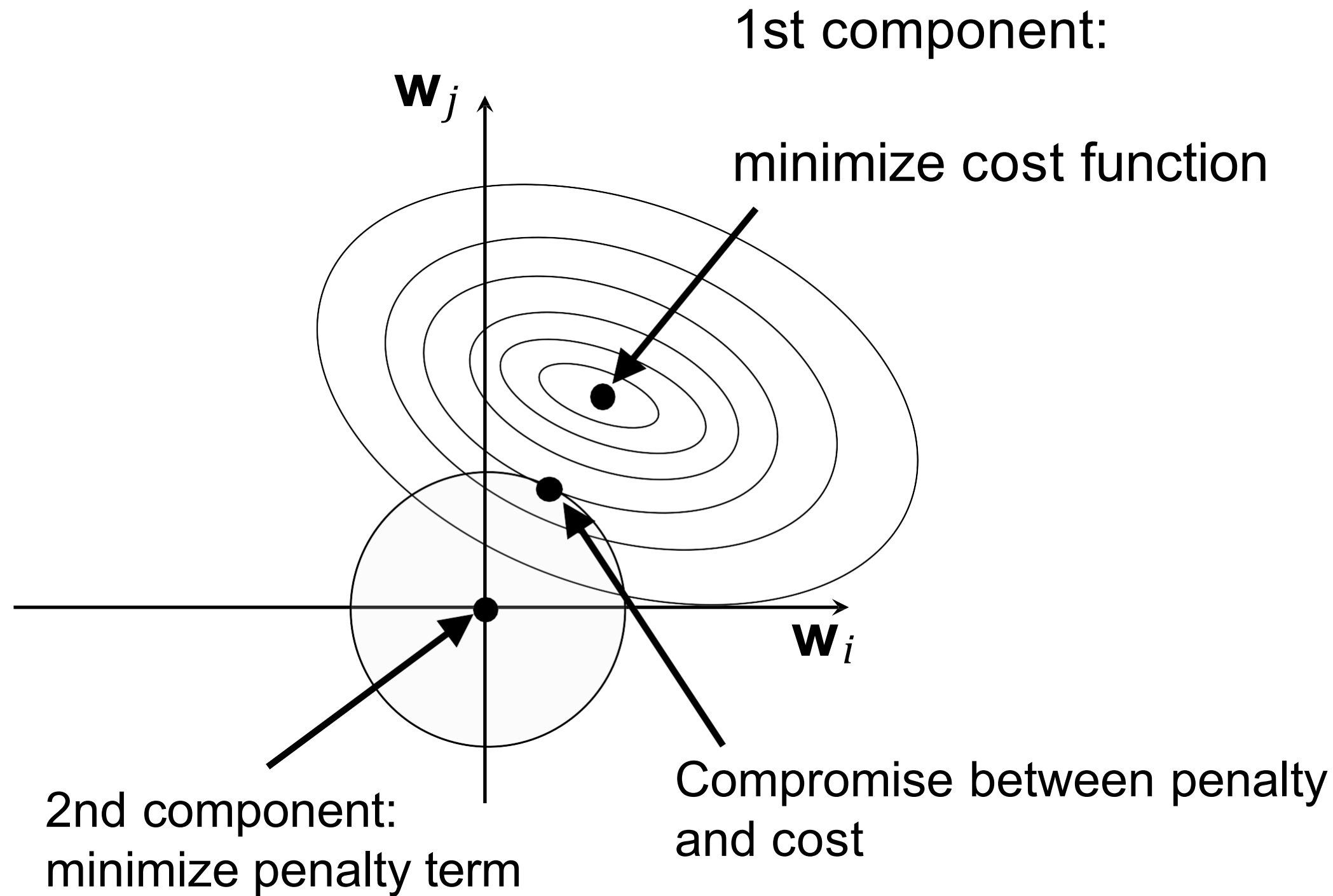
$$\text{Cost}_{\mathbf{w}, \mathbf{b}} = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y^{[i]}, \hat{y}^{[i]})$$

$$\text{L2-Regularized-Cost}_{\mathbf{w}, \mathbf{b}} = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y^{[i]}, \hat{y}^{[i]}) + \frac{\lambda}{n} \sum_j w_j^2$$

where: $\sum_j w_j^2 = \|\mathbf{w}\|_2^2$

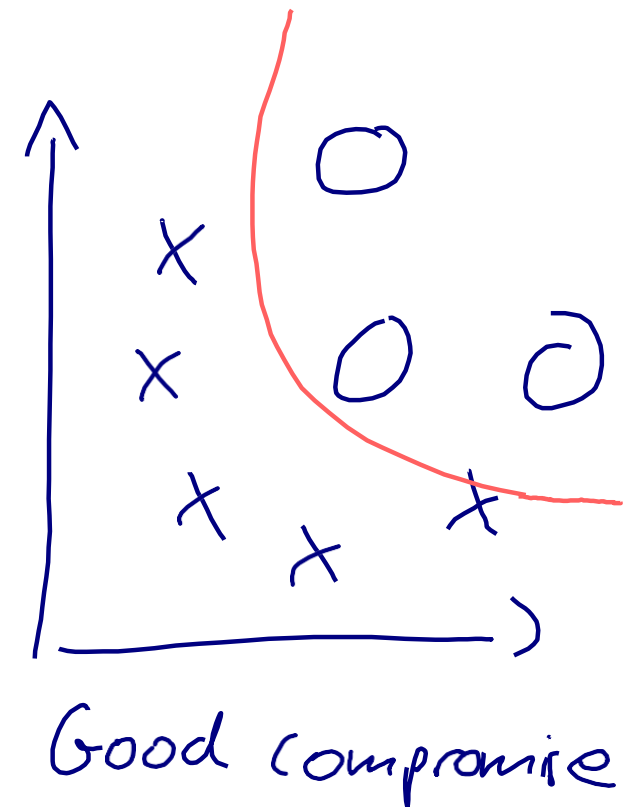
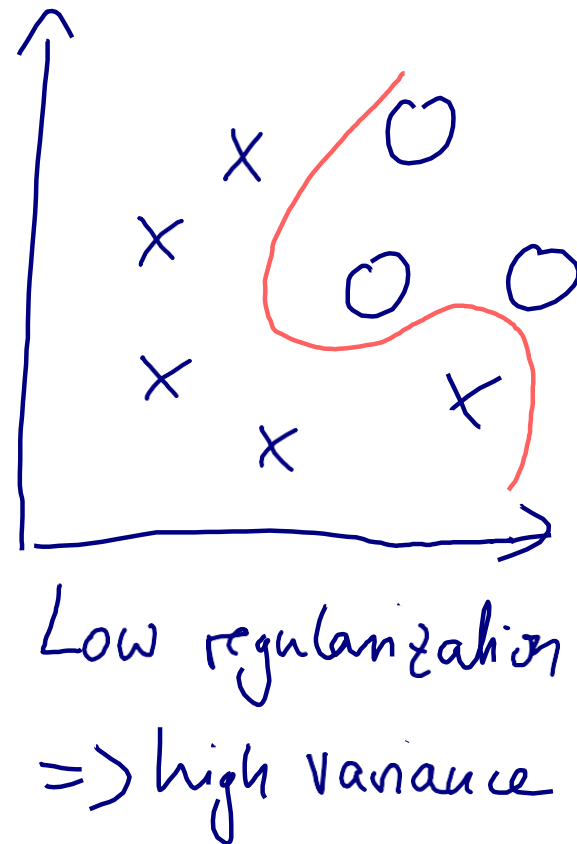
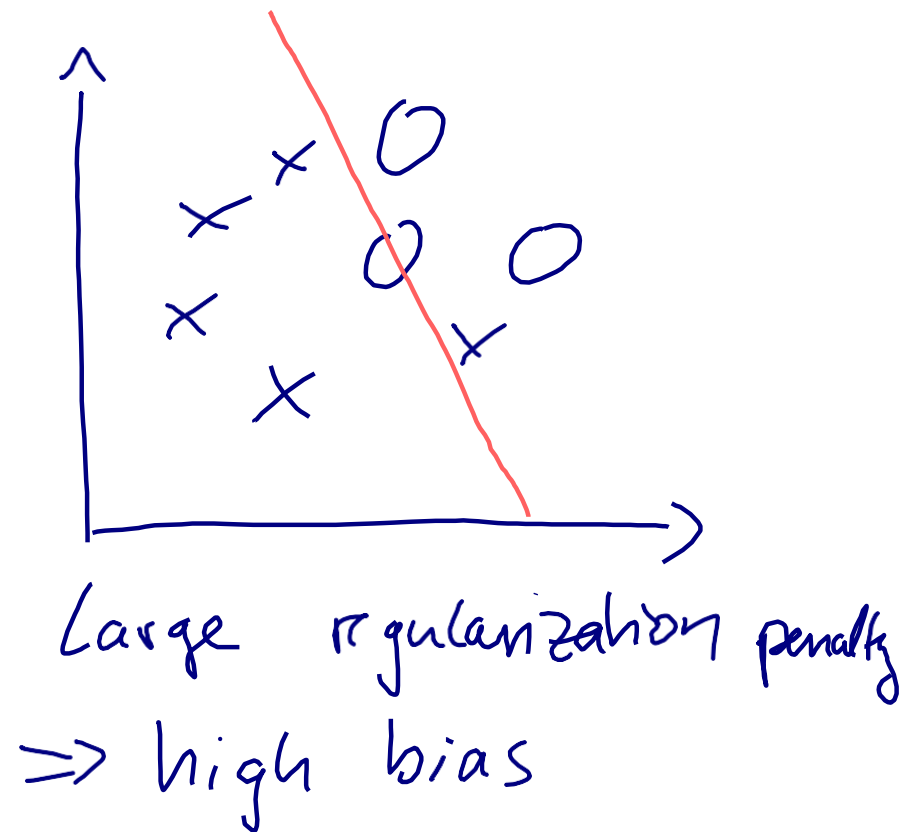
and λ is a hyperparameter

Geometric Interpretation of L_2 Regularization



Effect of Norm Penalties on the Decision Boundary

Assume a nonlinear model



L₂ Regularization for Neural Nets

Regular gradient descent update:

$$w_{i,j} := w_{i,j} - \eta \frac{\partial \mathcal{L}}{\partial w_{i,j}}$$

Gradient descent update with L2 regularization:

$$w_{i,j} := w_{i,j} - \eta \left(\frac{\partial \mathcal{L}}{\partial w_{i,j}} + \frac{2\lambda}{n} w_{i,j} \right)$$

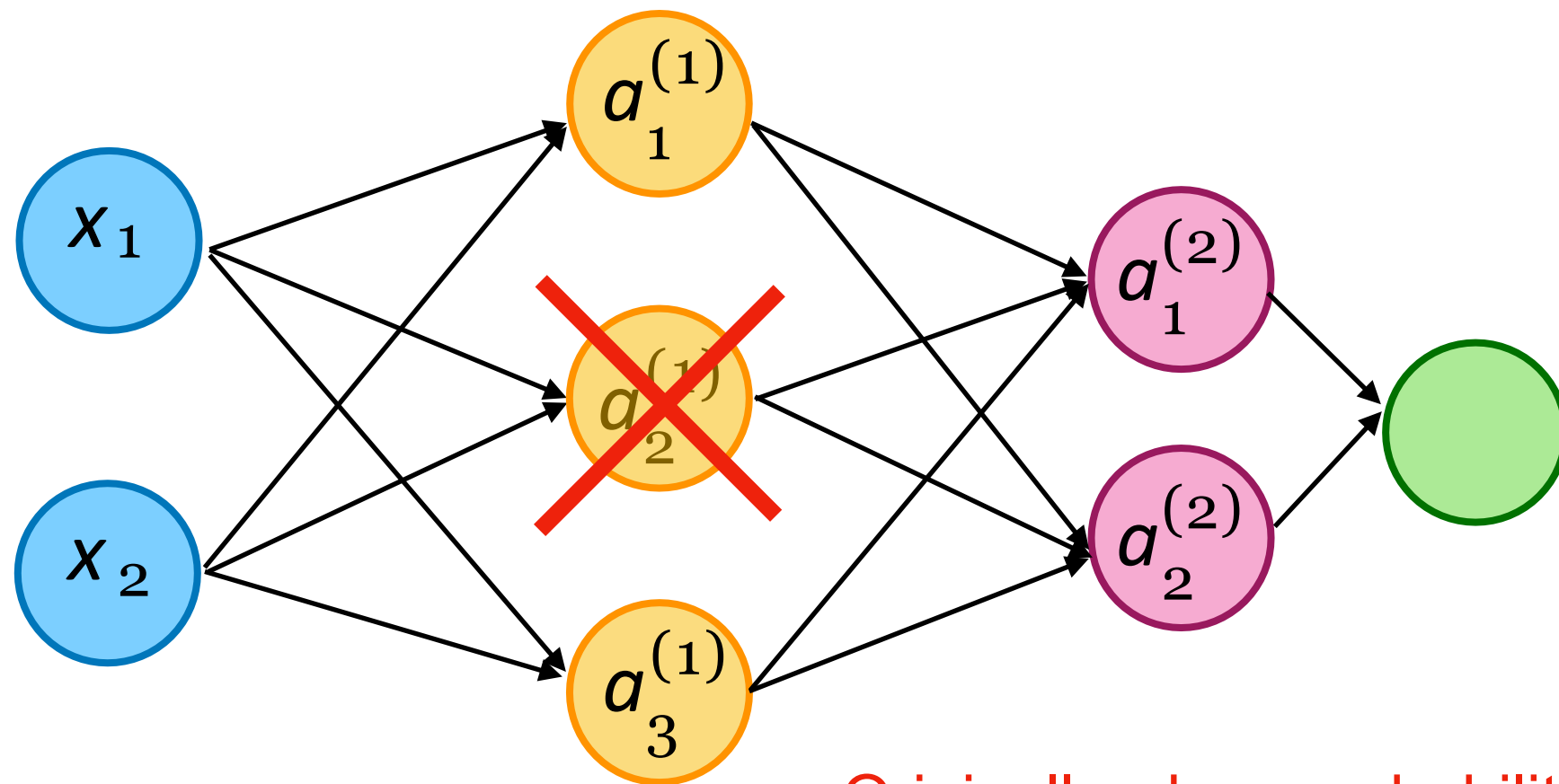
Dropout

Original research articles:

Hinton, G. E., Srivastava, N., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2012). Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1), 1929-1958.

Dropout in a Nutshell: Dropping Nodes



Originally, drop probability 0.5

(but 0.2-0.8 also common now)

Dropout in a Nutshell: Dropping Nodes

How do we drop the nodes practically/efficiently?

Bernoulli Sampling (during training):

- $p :=$ drop probability
- $\mathbf{v} :=$ random sample from uniform distribution in range $[0, 1]$
- $\forall i \in \mathbf{v} : v_i := 0$ if $v_i < p$ else 1
- $\mathbf{a} := \mathbf{a} \odot \mathbf{v}$ ($p \times 100\%$ of the activations $\mathbf{a}$ will be zeroed)

Then, after training when making predictions (during "inference")

scale activations via $\mathbf{a} := \mathbf{a} \odot (1 - p)$

Q for you: Why is this required?

Dropout: Co-Adaptation Interpretation

Why does Dropout work well?

- Network will learn not to rely on particular connections too heavily
- Thus, will consider more connections (because it cannot rely on individual ones)
- The weight values will be more spread-out (may lead to smaller weights like with L2 norm)
- Side note: You can certainly use different dropout probabilities in different layers (assigning them proportional to the number of units in a layer is not a bad idea, for example)

Dropout: Ensemble Method Interpretation

- In dropout, we have a "different model" for each minibatch
- Via the minibatch iterations, we essentially sample over $M=2^h$ models, where h is the number of hidden units
- Restriction is that we have weight sharing over these models, which can be seen as a form of regularization
- During "inference" we can then average over all these models (but this is very expensive)

Dropout: Ensemble Method Interpretation

- During "inference" we can then average over all these models (but this is very expensive)

This is basically just averaging log likelihoods (this is for one particular class):

$$p_{\text{Ensemble}} = \left[\prod_{j=1}^M p^{\{i\}} \right]^{1/M} = \exp \left[1/M \sum_{j=1}^M \log(p^{\{i\}}) \right]$$

(you may know this as the "geometric mean" from other classes)

For multiple classes, we need to normalize so that the probas sum

to 1:

$$p_{\text{Ensemble}, j} = \frac{p_{\text{Ensemble}, j}}{\sum_{j=1}^k p_{\text{Ensemble}, j}}$$

Dropout: Ensemble Method Interpretation

- During "inference" we can then average over all these models (but this is very expensive)
- However, using the last model after training and scaling the predictions by a factor $1-p$ approximates the geometric mean and is much cheaper
(actually, it's exactly the geometric mean if we have a linear model)

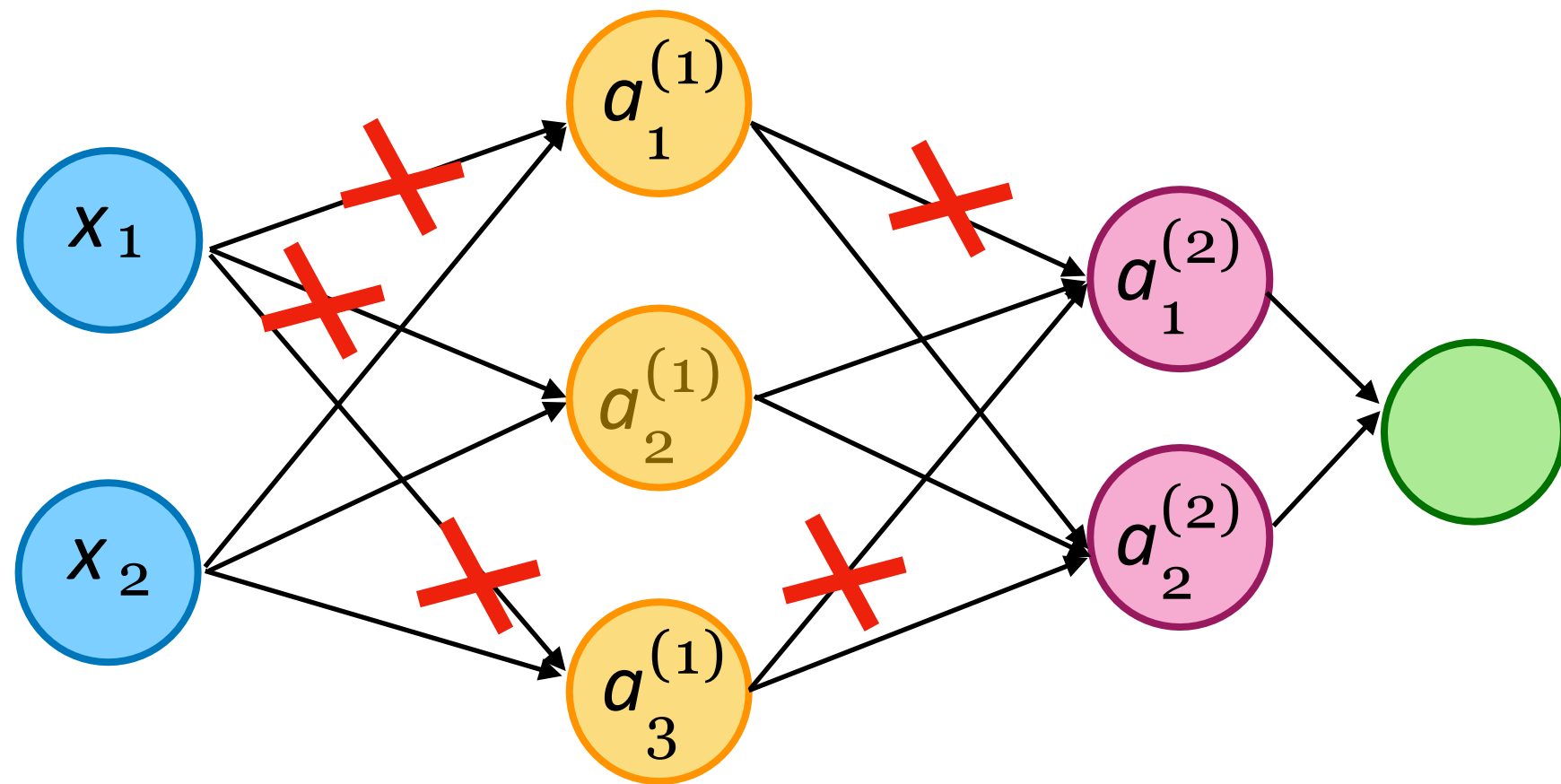
Inverted Dropout

- Most frameworks implement inverted dropout
- Here, the activation values are scaled by the factor $(1-p)$ during training instead of scaling the activations during "inference"
- I believe Google started this trend (because it's computationally cheaper in the long run if you use your model a lot after training)
- PyTorch's Dropout implementation is also inverted Dropout

Dropout: More Practical Tips

- Don't use Dropout if your model does not overfit
- However, in that case above, it is then recommended to increase the capacity to make it overfit, and then use dropout to be able to use a larger capacity model (but make it not overfit)

DropConnect: Randomly Dropping Weights



DropConnect

- Generalization of Dropout
- More "possibilities"
- Less popular & doesn't work so well in practice

Original research article:

Wan, L., Zeiler, M., Zhang, S., Le Cun, Y., & Fergus, R. (2013, February). Regularization of neural networks using DropConnect. In *International conference on machine learning* (pp. 1058-1066).

Recommended Reading Assignment

- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1), 1929-1958.
<http://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf>

